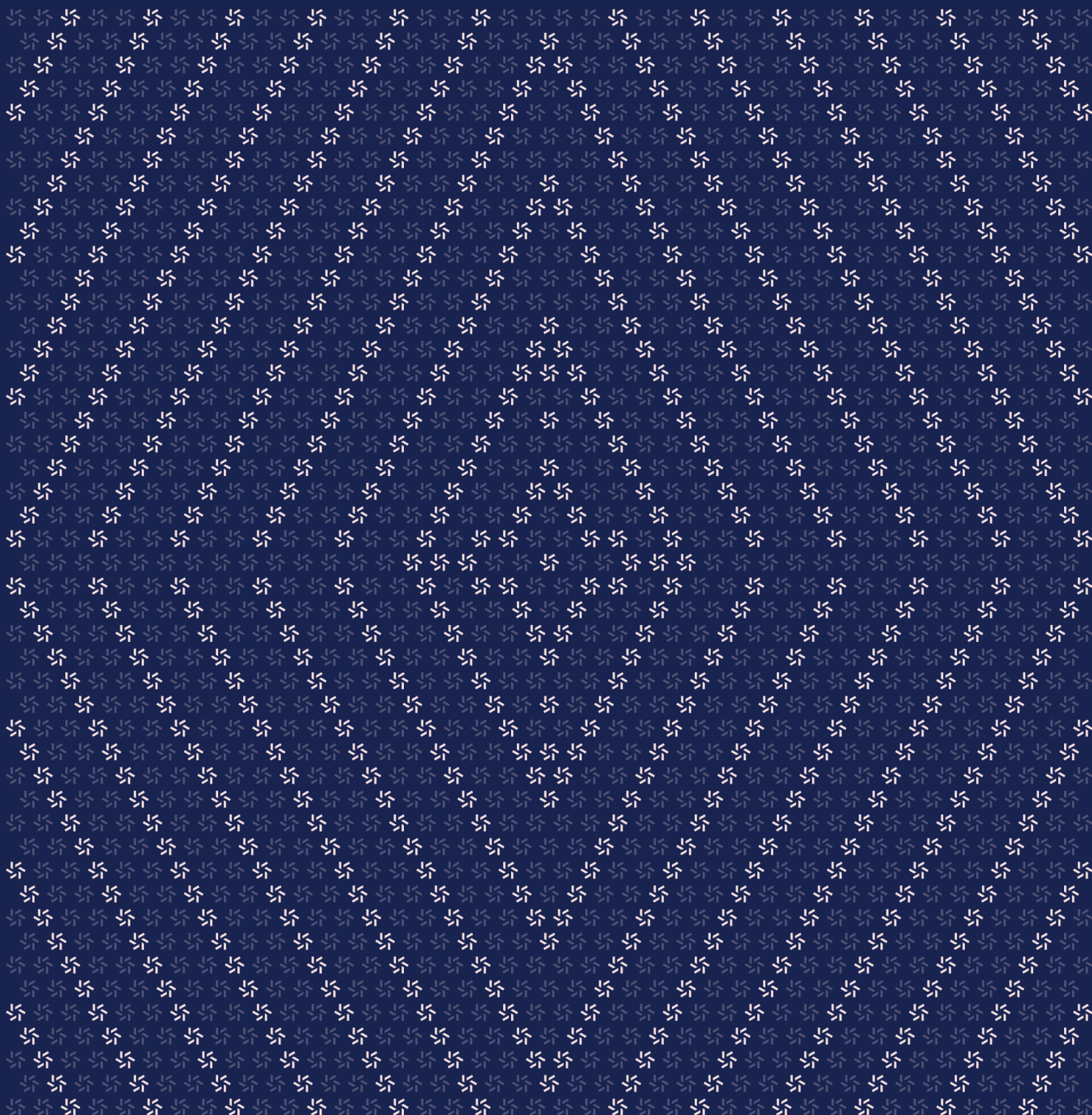


March 16, 2026

FSP V1 and V2

Smart Contract Security Assessment



Contents

About Zellic	5
1. Overview	5
1.1. Executive Summary	6
1.2. Goals of the Assessment	6
1.3. Non-goals and Limitations	6
1.4. Results	6
2. Introduction	7
2.1. About FSP V1 and V2	8
2.2. Methodology	8
2.3. Scope	10
2.4. Project Overview	11
2.5. Project Timeline	12
3. Detailed Findings	12
3.1. Feed value could overflow in <code>_applySubmitted</code>	13
3.2. The <code>submitUpdates</code> function validates sortition against the current reward epoch instead of the sortition block's reward epoch	15
3.3. Cross-chain signature replay in voter registration due to missing <code>chainId</code> in message hash	17
3.4. Replayable governance fee updates in <code>governanceFeeSetup</code>	19
3.5. No refund mechanism for overpayment in <code>fetchCurrentFeeds</code>	21
3.6. Arithmetic overflow risk in Inflation contract migration	23

3.7.	Missing validation in <code>fetchCurrentFeeds</code> causes users to pay fees for inactive indexes	25
3.8.	Eager evaluation of revert-message decoding can brick <code>submitAndPass</code>	27
3.9.	Getters return potentially nonfinal values for the mutable scheduled epoch	29
3.10.	The <code>relay</code> function accepts <code>bytes32(0)</code> as a valid Merkle root	31
3.11.	Silent overflow in <code>mulDivRoundUp</code>	33
3.12.	Missing initial checkpoint for partially zero public keys	35
3.13.	The <code>checkExecutorAndAllowedRecipient</code> function treats <code>address(0)</code> as a valid delegation account when none exists	37
3.14.	Missing PDA status check allows claims to disabled PDA	40
3.15.	Nonstandard initialization pattern	42
3.16.	The <code>requestMinting</code> function does not validate the cumulative pending amount against <code>maxMintingRequestWei</code>	43
3.17.	The 30/360 day-count convention causes effective calendar-year inflation to exceed stated annual rate	45
3.18.	Unhandled return value of <code>daemonize()</code>	47
3.19.	The <code>clear</code> function does not deduplicate same-block checkpoints	49
3.20.	Zero-value transfer during initial VPToken setup can force replacement-mode VPCContract	51
4.	Discussion	53
4.1.	Relay migration risk in RandomNumberV2 integrations	54
5.	System Design	54
5.1.	Governance	55
5.2.	Inflation	56
5.3.	FlareDaemon	58

5.4.	ClaimSetupManager	60
5.5.	Supply	62
5.6.	Fast updates	63
5.7.	Flare Time Series Oracle (FTSO)	66
5.8.	Flare Data Connector (FDC)	68
5.9.	EntityManager	70
5.10.	Relay	71
5.11.	RewardManager	74
5.12.	FlareSystemsManager	76

6.	Assessment Results	78
6.1.	Disclaimer	79

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the [#1 CTF \(competitive hacking\) team](#) worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io and follow [@zellic_io](#) on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io.



1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Flare Network from November 24th, 2025 to February 25th, 2026. During this engagement, Zellic reviewed FSP V1 and V2's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following question:

- Is it possible for a malicious user to manipulate the voting power?
-

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Infrastructure relating to the project
- Key custody

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

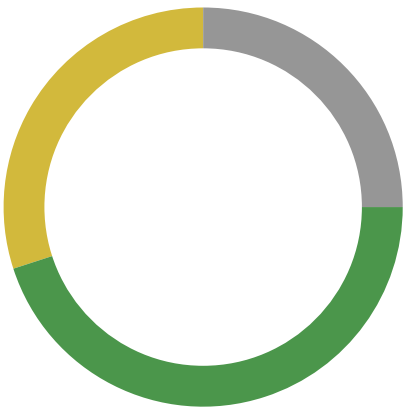
1.4. Results

During our assessment on the scoped FSP V1 and V2 contracts, we discovered 20 findings. No critical issues were found. Six findings were of medium impact, nine were of low impact, and the remaining findings were informational in nature.

Additionally, Zellic recorded its notes and observations from the assessment for the benefit of Flare Network in the Discussion section ([4.7](#)).

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	0
Medium	6
Low	9
Informational	5



2. Introduction

2.1. About FSP V1 and V2

Flare Network contributed the following description of FSP V1 and V2:

The Flare Systems Protocol (FSP) is a foundational infrastructure designed to support Flare's enshrined protocols (technically referred to as sub-protocols). Its primary goal is to facilitate secure, efficient, and decentralized consensus mechanisms through weighted voting by a select group of entities known as data providers or voters. These data providers are off-chain participants who accrue vote power from the Flare community via delegations of wrapped FLR tokens (WFLR) or stakes. FSP ensures that agreements on offchain data or calculations are reached securely and fairly, enabling the reliable operation of sub-protocols like the Flare Time Series Oracle and the Flare Data Connector.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the contracts.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability

weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

Finally, Zellic provides a list of miscellaneous observations that do not have security impact or are not directly related to the scoped contracts itself. These observations — found in the Discussion (4. 7) section of the document — may include suggestions for improving the codebase, or general recommendations, but do not necessarily convey that we suggest a code change.

2.3. Scope

The engagement involved a review of the following targets:

FSP V1 and V2 Contracts

Type	Solidity
Platform	EVM-compatible
Target	flare-smart-contracts-v1
Repository	https://github.com/flare-foundation/flare-smart-contracts-v1
Version	e670b1fbe4e984c4a169a15cb930c2b2446fb952
Programs	claiming/implementation/*.sol genesis/implementation/FlareDaemon.sol governance/implementation/Governed.sol governance/implementation/GovernedAtGenesis.sol governance/implementation/GovernedBase.sol inflation/implementation/Inflation.sol inflation/implementation/InflationAllocation.sol inflation/implementation/Supply.sol inflation/lib/*.sol utils/implementation/AddressSet.sol utils/implementation/SafePct.sol utils/implementation/GovernedAndFlareDaemonized.sol token/implementation/Checkpointable.sol token/implementation/CleanupBlockNumberManager.sol token/implementation/Delegatable.sol token/implementation/VPCContract.sol token/implementation/VPToken.sol token/implementation/WNat.sol token/lib/*.sol

Target	flare-smart-contracts-v2
Repository	https://github.com/flare-foundation/flare-smart-contracts-v2 ↗
Version	e71bafef8a8b99d7f08afad574e1461649ecf86
Programs	customFeeds/implementation/SFlrCustomFeed.sol fastUpdates/implementation/*.sol fastUpdates/lib/*.sol fdc/implementation/*.sol fscV1/implementation/*.sol ftso/implementation/*.sol inflation/implementation/*.sol governance/implementation/Governed.sol governance/implementation/GovernedBase.sol governance/implementation/GovernedProxyImplementation.sol protocol/implementation/EntityManager.sol protocol/implementation/FlareSystemsCalculator.sol protocol/implementation/FlareSystemsManager.sol protocol/implementation/FtsoV2.sol protocol/implementation/FtsoV2Proxy.sol protocol/implementation/NodePossessionVerifier.sol protocol/implementation/RewardManager.sol protocol/implementation/RewardOffersManagerBase.sol protocol/implementation/Submission.sol protocol/implementation/VoterPreRegistry.sol protocol/implementation/VoterRegistry.sol protocol/implementation/WNatDelegationFee.sol protocol/implementation/Relay.sol protocol/lib/*.sol utils/implementation/AddressUpdatable.sol utils/implementation/TokenPoolBase.sol utils/lib/*.sol

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 16.6 person-weeks. The assessment was conducted by two consultants over the course of 11 calendar weeks.

Contact Information

The following project manager was associated with the engagement:

Jacob Goreski
✈ Engagement Manager
jacob@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Kritsada Dechawattana
✈ Engineer
kritsada@zellic.io ↗

Weipeng Lai
✈ Engineer
weipeng.lai@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

November 24, 2025	Start of primary review period
--------------------------	--------------------------------

November 25, 2025	Kick-off call
--------------------------	---------------

February 25, 2026	End of primary review period
--------------------------	------------------------------

3. Detailed Findings

3.1. Feed value could overflow in `_applySubmitted`

Target	FastUpdater		
Category	Coding Mistakes	Severity	High
Likelihood	Low	Impact	Medium

Description

FastUpdater stores feed values as eight 32-bit entries packed per bytes32 slot. The `_adjustDecimals` function normalizes each feed value to strictly below 2^{29} , but this normalization only runs at reward-epoch boundaries or when a feed is reset — not during per-block delta application.

When `FastUpdater::_applySubmitted()` processes a "mul" delta, it multiplies the current 32-bit feed value by `scale` (a 1x127 fixed-point value slightly above 1.0) and right-shifts by 127:

```
if eq(deltaReduced, 1) {
  // mul
  feedReduced := mul(feedReduced, scale)
  feedReduced := shr(127, feedReduced)
```

Because the normalized value is strictly below 2^{29} , there are exactly three bits of headroom before exceeding the 32-bit boundary. A cumulative price increase exceeding $8 \times (2^3)$ within a single reward epoch is therefore possible to cause overflow. Once the result exceeds 32 bits, the value is packed back without any clamp:

```
feedReduced := shl(sub(224, mul(1, 32)), feedReduced)
newFeed := or(newFeed, feedReduced)
```

For feed position `l = 0`, the overflow bit is silently truncated, causing the price to wrap to an incorrect value. For positions `l = 1..7`, the excess bits additionally land inside the adjacent feed's 32-bit range via the `or`, corrupting its stored value as well.

Impact

The primary impact is that a feed's price silently overflows and wraps to an incorrect value. Any downstream consumer of `fetchCurrentFeeds` would receive a drastically incorrect price. As a secondary effect, the overflow bits corrupt the adjacent feed packed in the same storage slot, potentially producing incorrect prices for an unrelated feed as well.

Recommendations

Clamp `feedReduced` to `type(uint32).max` after the multiply-shift so that an overflowed price saturates at the maximum representable value rather than wrapping to a drastically incorrect one:

```
// mul
feedReduced := mul(feedReduced, scale)
feedReduced := shr(127, feedReduced)
if gt(feedReduced, 0xFFFFFFFF) {
    feedReduced := 0xFFFFFFFF
}
```

Remediation

This issue has been acknowledged by Flare Network.

Flare Network provided the following response to this finding:

The impact is limited, as it depends on an extremely unlikely scenario—an 8× price change within 3.5 days—especially given that our price feeds are highly stable in practice. Considering this low probability and the difficulty of upgrading the contract, we have decided not to pursue changes.

3.2. The submitUpdates function validates sortition against the current reward epoch instead of the sortition block's reward epoch

Target	FastUpdater		
Category	Coding Mistakes	Severity	Medium
Likelihood	High	Impact	Medium

Description

In `FastUpdater::submitUpdates`, the `SortitionState` used to verify a provider's `SortitionCredential` is constructed using `currentRewardEpochId`, which corresponds to the reward epoch at the time the transaction is executed (i.e., at `block.number`).

```
function submitUpdates(FastUpdates calldata _updates) external {
    // [...]
    uint256 rewardEpochId = currentRewardEpochId;
    // [...]
    (G1Point memory key, uint256 weight) = _providerData(signingPolicyAddress,
        rewardEpochId);
    SortitionState memory sortitionState = SortitionState({
        baseSeed: flareSystemsManager.getSeed(rewardEpochId),
        blockNumber: _updates.sortitionBlock,
        scoreCutoff: thresholds[_updates.sortitionBlock % (sw + 1)],
        weight: weight,
        pubKey: key
    });
    // [...]
}
```

However, the sortition credential is generated for a specific `_updates.sortitionBlock`, and the correct `SortitionState` should use the reward epoch that contains `_updates.sortitionBlock`, not the epoch at the time of submission.

Because submissions are allowed for past blocks within `submissionWindow`, `_updates.sortitionBlock` may belong to the previous reward epoch while `currentRewardEpochId` already points to the new one. In that case, sortition is verified with mismatched epoch parameters.

Impact

There are two main impacts.

1. **Legitimate submissions can be rejected.** A provider with a valid credential for block `b` in epoch `r` can still be inside the submission window after the chain enters epoch `r+1`. Their valid epoch-`r` credential may fail because verification uses epoch-`r+1` seed/weight/key.
2. **Unauthorized submissions can be accepted.** A provider can generate a credential that satisfies verification under epoch-`r+1` parameters for a `sortitionBlock` that actually belongs to epoch `r`. This allows acceptance of credentials that are invalid for the sortition block's real epoch.

Recommendations

Determine the reward epoch ID corresponding to `_updates.sortitionBlock` and use it to construct the `SortitionState`, rather than using `currentRewardEpochId`.

Remediation

This issue has been acknowledged by Flare Network.

Flare Network provided the following response to this finding:

This behavior is intentional. To optimize gas usage, we deliberately chose not to include the reward epoch ID alongside the `_updates` parameter.

3.3. Cross-chain signature replay in voter registration due to missing chainId in message hash

Target	VoterRegistry		
Category	Business Logic	Severity	High
Likelihood	Low	Impact	Medium

Description

In `VoterRegistry::registerVoter`, the message hash used for signature verification is computed only from the reward epoch ID and the voter address:

```
function registerVoter(address _voter, Signature calldata _signature)
    external {
        (uint32 rewardEpochId, IEntityManager.VoterAddresses
        memory voterAddresses) = _getRegistrationData(_voter);
        // check signature
        bytes32 messageHash = keccak256(abi.encode(rewardEpochId, _voter));
        bytes32 signedMessageHash
        = MessageHashUtils.toEthSignedMessageHash(messageHash);
        address signingPolicyAddress = ECDSA.recover(signedMessageHash,
        _signature.v, _signature.r, _signature.s);
        require(signingPolicyAddress == voterAddresses.signingPolicyAddress,
        "invalid signature");
        // register voter
        _registerVoter(_voter, rewardEpochId, _signature, voterAddresses);
    }
```

Because the hash does *not* include `block.chainid`, the resulting signature is not bound to a specific chain. As a result, a signature produced on one network can be replayed on another network where the same `(rewardEpochId, _voter)` tuple and `signingPolicyAddress` are valid.

Impact

If a data provider reuses the same voter identity and signing-policy addresses across two Flare networks (e.g., Songbird and Flare mainnet), and a given `rewardEpochId` has begun on one network and is about to begin on the other, an attacker can obtain the registration transaction from one network, then submit the same signature on the other network to register the provider for that reward epoch without their intent.

Potential consequences include the following:

- **Missed signing duties and penalties.** The provider may be unaware of the registration on the target network, leading to missed signing obligations and potential penalties.
- **Displacement of legitimate voters.** If the voter set is at capacity (`maxVoters`), a replayed registration may trigger the replacement logic in `_registerVoter`, potentially evicting a lower-weight voter who intended to participate.
- **Potential finalization delays.** If enough high-weight voters are forcibly registered but do not participate, the signing policy may struggle to reach threshold, degrading liveness and potentially delaying finalization.

Recommendations

Bind signatures to a specific chain by including `block.chainid` in the signed message:

```
bytes32 messageHash = keccak256(abi.encode(rewardEpochId, _voter));  
bytes32 messageHash = keccak256(abi.encode(block.chainid, rewardEpochId, _  
voter));
```

Apply the same mitigation to `VoterPreRegistry::preRegisterVoter`, which uses the same message-hash construction.

Remediation

This issue has been acknowledged by Flare Network, and a fix was implemented in commit [a122bc4e](#).

3.4. Replayable governance fee updates in governanceFeeSetup

Target	Relay		
Category	Business Logic	Severity	Medium
Likelihood	Low	Impact	Medium

Description

The `governanceFeeSetup` function in Relay accepts signed governance updates and applies new `protocolFeeInWei` values after verifying signatures via `_verifyCustomSignature`. Internally, signature verification calls `address(this).call(_relayMessage)`, which executes `relay` with `protocolId == 1` and returns `(messageHash, rewardEpochId)`.

Although `relay` performs an "already relayed" check against `merkleRootsPrivate`, the `protocolId == 1` branch returns early before writing to that mapping. As a result, the signed relay payload is never marked as consumed and can be submitted repeatedly.

The `governanceFeeSetup` function itself has no nonce or deduplication tracking for governance configs. Its only freshness gate is `returnRewardEpochId == lastInitializedRewardEpoch` or `returnRewardEpochId == lastInitializedRewardEpoch - 1`.

Therefore, any previously executed governance fee update remains replayable for up to two signing-policy epochs. Consider the following scenario:

1. Governance applies fee config A in epoch `v`.
2. Governance applies fee config B in epoch `v` or `v+1`, overriding A.
3. An attacker extracts the calldata for A from on-chain transaction history and replays it.
4. Then `protocolFeeInWei` is reverted from B back to A.

Impact

Governance cannot reliably enforce fee updates for up to two reward epochs after a change. Within that window, any party can roll back fees to previously signed values, causing protocol revenue loss (if replaying lower fees) or user overpayment (if replaying higher fees).

Recommendations

Add explicit replay protection to governance fee updates. One approach is to include a monotonic nonce field in `RelayGovernanceConfig`, incorporate it into the signed hash, and require that each invocation strictly increments over the last recorded nonce on chain.

Remediation

This issue has been acknowledged by Flare Network.

Flare Network provided the following response to this finding:

This vulnerability is not a major issue because the replay window is strictly limited to two reward epochs and requires highly unlikely, back-to-back fee changes.

3.5. No refund mechanism for overpayment in fetchCurrentFeeds

Target	FastUpdater		
Category	Business Logic	Severity	Medium
Likelihood	Low	Impact	Medium

Description

The `FastUpdater::fetchCurrentFeeds` function is payable and dynamically calculates the required fee via `feeCalculator.calculateFeeByIndices`. While the function correctly enforces that the user sends enough funds (`msg.value >= fee`), it only forwards the exact fee amount to the `feeDestination`:

```
function fetchCurrentFeeds(uint256[] calldata _indices)
    external payable
    returns (
        uint256[] memory _feeds,
        int8[] memory _decimals,
        uint64 _timestamp
    )
{
    // calculate fees
    if (freeFetchAddresses.index[msg.sender] == 0) {
        uint256 fee = feeCalculator.calculateFeeByIndices(_indices);
        require(msg.value >= fee, "too low fee");
        if (msg.value > 0) {
            (bool success, ) = feeDestination.call{value: fee}("");
            require(success, "fee transfer failed");
        }
    } // [...]
}
```

Any `msg.value` exceeding the calculated fee remains trapped in the `FastUpdater` contract. Because the contract provides no governance-controlled withdrawal mechanism, this surplus is permanently locked.

Additionally, a secondary flaw exists in the conditional logic: the fee transfer is triggered by `msg.value > 0` rather than `fee > 0`. If the calculated fee is 0 but a caller still sends native tokens, the contract executes a zero-value transfer (`feeDestination.call{value: 0}("")`) and absorbs the caller's entire `msg.value`.

Impact

Callers who send more native tokens than the exact required fee will have the excess permanently locked in the contract, with no available recovery path.

Recommendations

Refund excess funds to an explicit refundTo address parameter supplied by the caller, and tighten the transfer guard to `fee > 0`:

```
uint256 fee = feeCalculator.calculateFeeByIndices(_indices);
require(msg.value >= fee, "too low fee");
if (msg.value > 0) {
  if (fee > 0) {
    (bool success, ) = feeDestination.call{value: fee}("");
    require(success, "fee transfer failed");
  }
  uint256 excess = msg.value - fee;
  if (excess > 0) {
    (bool success, ) = refundTo.call{value: excess}("");
    require(success, "refund failed");
  }
}
```

Remediation

This issue has been acknowledged by Flare Network.

Flare Network provided the following response to this finding:

Since the impact depends completely on user error and since the contract is difficult to update, we have decided not to amend it.

3.6. Arithmetic overflow risk in Inflation contract migration

Target	Inflation		
Category	Coding Mistakes	Severity	Medium
Likelihood	Low	Impact	Medium

Description

The `Inflation::setInitialData()` function migrates historical inflation data from an older Inflation contract version (`IIInflationV1`) to the new contract. This migration process involves accumulating multiple inflation-related values across reward services and annums. The vulnerability lies in the use of unchecked arithmetic operations, which can lead to integer overflow in Solidity 0.7.6 that lacks built-in overflow protection. Specifically, the function accumulates values in nested loops without using the `SafeMath` library.

```
function setInitialData(
    IIInflationV1 _oldInflation,
    uint256 _noOfAnnums
)
external
onlyImmediateGovernance
{
    [...]
    for (uint256 i = 0; i < _noOfAnnums; i++) {
        IIInflationV1.InflationAnnumState memory annum
        = _oldInflation.getAnnum(i);
        for (uint256 j = 0; j < annum.rewardServices.rewardServices.length;
            j++) {
            [...]
            rewardService.lastDailyAuthorizedInflationWei
            = oldRewardService.lastDailyAuthorizedInflationWei;
            rewardService.authorizedInflationWei += oldRewardService.
                authorizedInflationWei;
            rewardService.inflationTopupRequestedWei += oldRewardService.
                inflationTopupRequestedWei;
            rewardService.inflationTopupDistributedWei += oldRewardService.
                inflationTopupReceivedWei;

            totalAuthorizedInflationWeiTemp += oldRewardService.authorizedInfla
```

```
        tionWei;

        totalInflationTopupRequestedWeiTemp += oldRewardService.inflationTo
        pupRequestedWei;
        totalInflationTopupDistributedWeiTemp += oldRewardService.
        inflationTopupReceivedWei;
    }
}
[...]
```

```
inflationTimeSlot.endTimeStamp = lastAuthorizationTs + AUTHORIZE_TIME_
FRAME_SEC - 1;
}
```

Impact

A silent overflow in `totalAuthorizedInflationWeiTemp` would cause `totalAuthorizedInflationWei` and `totalRecognizedInflationWei` to be written with a drastically underflowed value. Because every subsequent daily inflation authorization is gated on these totals, an underflowed `totalRecognizedInflationWei` would cause `_totalRecognizedInflationWei.sub(_totalAuthorizedInflationWei)` to revert (since `authorized > recognized`), permanently bricking the `daemonize()` function and halting all future inflation authorization and minting. Since `daemonize()` is called by `FlareDaemon` on every block, this would also trigger `FlareDaemon`'s error-handling fallback path indefinitely.

Recommendations

Replace all `+=` accumulations in `setInitialData()` with `SafeMath.add()`, matching the pattern used everywhere else in the contract. This aligns the migration function with the contract's existing defensive coding standard and ensures any overflow is caught with a revert rather than silently corrupting the contract's core accounting state.

Remediation

This issue has been acknowledged by Flare Network.

Flare Network provided the following response to this finding:

Overflow is a non-issue due to the 5B/year inflation ceiling. We skipped the check for gas optimization after verifying that the new contract's state perfectly mirrors the old one. All figures were cross-referenced off-chain before the governance update was pushed.

3.7. Missing validation in `fetchCurrentFeeds` causes users to pay fees for inactive indexes

Target	FastUpdater		
Category	Business Logic	Severity	Medium
Likelihood	Low	Impact	Low

Description

When a feed is removed via `FastUpdatesConfiguration::removeFeeds`, its corresponding `feedConfigurations` entry is deleted and its index is added to `unusedIndices`. This process also triggers `FastUpdater::removeFeeds`, which zeroes out both the feed's value and its decimals.

```
function removeFeeds(bytes21[] calldata _feedIds) external onlyGovernance {
    uint256[] memory indices = new uint256[](_feedIds.length);
    for (uint256 i = 0; i < _feedIds.length; i++) {
        uint256 index = feedIdToIndex[_feedIds[i]];
        require(index != 0, "feed does not exist");
        index--;
        indices[i] = index;
        unusedIndices.push(index);
        delete feedConfigurations[index];
        delete feedIdToIndex[_feedIds[i]];
        emit FeedRemoved(_feedIds[i], index);
    }
    fastUpdater.removeFeeds(indices);
}
```

However, this removal process does not automatically clear the fee in `FeeCalculator` for the removed indexes. Consequently, if a removed index has a nonzero fee, any subsequent calls to `FastUpdater::fetchCurrentFeeds` for that index will still charge the caller, despite returning empty data (`value = 0`, `decimals = 0`).

This also affects `FastUpdater::fetchAllCurrentFeeds`, which constructs the full index range `[0, 1, ..., length-1]` and forwards it to `fetchCurrentFeeds`, meaning callers are forced to pay fees for removed indexes.

Impact

Users calling `fetchCurrentFeeds` or `fetchAllCurrentFeeds` are unfairly charged fees for removed feeds, receiving meaningless `(0, 0)` data in return.

Additionally, downstream consumers that do not separately validate the feed ID against `FastUpdatesConfiguration` may interpret the zeroed response as legitimate price data, potentially leading to incorrect accounting in consuming protocols.

Recommendations

Implement validation within `fetchCurrentFeeds` to revert if a requested index corresponds to a removed feed. This can be achieved by verifying that the index maps to a nonzero feed ID:

```
bytes21 feedId = fastUpdatesConfiguration.getFeedId(_indices[i]);  
require(feedId != bytes21(0), "index not supported");
```

Additionally, update `fetchAllCurrentFeeds` to filter out removed indexes before calling `fetchCurrentFeeds`.

Remediation

This issue has been acknowledged by Flare Network.

Flare Network provided the following response to this finding:

```
Since the impact depends completely on wrong user action and since the contract is difficult  
to update, we have decided not to amend it.
```

3.8. Eager evaluation of revert-message decoding can brick submitAndPass

Target	Submission		
Category	Coding Mistakes	Severity	Low
Likelihood	Low	Impact	Low

Description

In Submission::submitAndPass, the require statement passes `_getRevertMsg(e)` as the error message:

```
function submitAndPass(bytes calldata _data) external returns (bool) {
    require(submitAndPassContract != address(0) && submitAndPassSelector !=
bytes4(0), "submitAndPass disabled");
    /* solhint-disable avoid-low-level-calls */
    //slither-disable-next-line arbitrary-send-eth
    (bool success, bytes memory e)
    = submitAndPassContract.call(bytes.concat(submitAndPassSelector, _data));
    /* solhint-enable avoid-low-level-calls */
    require(success, _getRevertMsg(e));

    return true;
}
```

In Solidity, function arguments are evaluated before the function itself is executed and require is evaluated just as any other function. This means that `_getRevertMsg(e)` executes regardless of whether success is true or false.

The `_getRevertMsg` helper assumes a standard `Error(string)` revert payload layout:

```
function _getRevertMsg(bytes memory _returnData)
    internal pure returns (string memory) {
    // If the _res length is less than 68, then the transaction failed silently
    (without a revert message)
    uint256 length = _returnData.length;
    if (length < 68) return "Transaction reverted silently";

    // solhint-disable-next-line no-inline-assembly
    assembly {
        _returnData := add(_returnData, 0x04) // Slice the signature hash
        mstore(_returnData, sub(length, 0x04)) // Set proper length
    }
```

```
    }  
    return abi.decode(_returnData, (string)); // All that remains is the revert  
    string  
}
```

However, when the external call succeeds, `e` contains ABI-encoded return data, not revert data. If the target function returns at least 68 bytes, `_getRevertMsg` will strip the first four bytes (treating them as a selector) and then attempt `abi.decode(..., (string))` on data that does not follow the `Error(string)` format.

This decoding can revert, causing `submitAndPass` to revert even though the underlying external call succeeded.

Impact

The `submitAndPassContract` and `submitAndPassSelector` are governance configurable via `setSubmitAndPassData`. As of the time of writing, they point to `FastUpdater::submitUpdates`, which returns no data, so the issue is not triggered.

However, if governance reconfigures these to a function with return data that cannot be decoded by `_getRevertMsg`, then `submitAndPass` will be blocked for that target.

Recommendations

Only decode revert data on failure. Replace `require` with an explicit `if` check:

```
(bool success, bytes memory e)  
    = submitAndPassContract.call(bytes.concat(submitAndPassSelector, _data));  
require(success, _getRevertMsg(e));  
if (!success) {  
    revert(_getRevertMsg(e));  
}
```

Remediation

This issue has been acknowledged by Flare Network.

Flare Network provided the following response to this finding:

Since upgrading the Submission contract requires a hard fork and there is currently no demonstrable impact, we are acknowledging this finding as an accepted risk without applying a fix.

3.9. Getters return potentially nonfinal values for the mutable scheduled epoch

Target	FtsoFeedDecimals, WNatDelegationFee, ClaimSetupManager		
Category	Business Logic	Severity	Low
Likelihood	Low	Impact	Low

Description

Three contracts follow a pattern where a setter schedules a parameter change for `currentRewardEpoch + offset`, and a corresponding getter accepts queries up to and including that same epoch. Because the setter can be called repeatedly during the current epoch to overwrite the scheduled value, the getter can return a result that is not yet final.

1. In `FtsoFeedDecimals`, `getDecimals` and `getDecimalsBulk` allow querying up to `current + decimalsUpdateOffset`, while `setDecimals` writes to that same epoch.
2. In `WNatDelegationFee`, `getVoterFeePercentage` allows querying up to `current + feePercentageUpdateOffset`, while `setVoterFeePercentage` writes to that same epoch.
3. In `ClaimSetupManager`, `getExecutorFeeValue` allows querying up to `current + feeValueUpdateOffset`, while `_updateExecutorFeeValue` writes to that same epoch.

All three contracts already expose a `get*Scheduled*Changes` function with a `_fixed` boolean that correctly distinguishes mutable from finalized entries. Additionally, the `IFtsoFeedDecimals` and `IWNatDelegationFee` interfaces include explicit `**NOTE**` comments warning that the value may still change for the boundary epoch. However, integrators who call only the simple getters without consulting the scheduled-changes functions or reading the interface notes could treat the returned value as final.

Impact

An integrator or downstream contract that queries any of these getters at the maximum-allowed epoch may cache or act on a value that is subsequently overwritten, leading to stale or inconsistent state in their own logic.

Recommendations

Tighten the upper-bound check in each getter from `<=` to `<` so that the still-mutable epoch cannot be queried through these functions.

Remediation

This issue has been acknowledged by Flare Network.

Flare Network provided the following response to this finding:

Since the contracts are non-upgradable and the demonstrable impact is very limited, we are acknowledging this finding as an accepted risk without applying a patch.

3.10. The relay function accepts bytes32(0) as a valid Merkle root

Target	Relay		
Category	Business Logic	Severity	High
Likelihood	Low	Impact	Low

Description

The relay function in Relay determines whether a (protocolId, votingRoundId) pair has already been finalized by checking if the stored Merkle root is nonzero:

```
// Check if merkleRootsPrivate[protocolId][votingRoundId] is set
// NOTE: M1 is already consumed. Hence using M_3 and M_4
mstore(add(memPtrGPO, M_3), protocolId) // key 1 (protocolId)
mstore(add(memPtrGPO, M_4), merkleRootsPrivate.slot) // merkleRoot slot
mstore(add(memPtrGPO, M_4), keccak256(add(memPtrGPO, M_3), 64))
mstore(add(memPtrGPO, M_3), votingRoundId) // key 2 (votingRoundId)

if gt(sload(keccak256(add(memPtrGPO, M_3), 64)), 0) {
    revertWithMessage(memPtrGPO, "Already relayed", 15)
}
```

The same zero value is the default for uninitialized storage slots. However, when storing the Merkle root from the signed message, relay writes it verbatim without rejecting bytes32(0):

```
// merkleRoot stored at merkleRootsPrivate[protocolId][votingRoundId]
sstore(
    keccak256(memPtrFor, 64),
    mload(add(memPtrFor, M_6_merkleRoot))
) // set Merkle Root
```

This creates a sentinel-value collision; a round finalized with merkleRoot == bytes32(0) is indistinguishable from a round that was never finalized.

While it is very unlikely in practice for a Merkle tree to produce a root of bytes32(0), it is not correct for the contract to rely on this cryptographic improbability as a security invariant when an explicit zero check would eliminate the issue unconditionally.

Impact

If a round's Merkle root is `bytes32(0)`, the “already relayed” guard never triggers. As a result, the same round can be finalized repeatedly within the `messageFinalizationWindowInRewardEpochs` window.

This is especially problematic on the `randomNumberProtocolId` path. Each successful relay unconditionally overwrites `stateData.randomVotingRoundId` and `stateData.isSecureRandom` without verifying that the new `votingRoundId` is greater than the current value. For example, if a newer round (e.g., round 101) has already been finalized, re-relaying a zero-root round (e.g., round 100) would roll `randomVotingRoundId` back to 100. Then, `getRandomNumber()` would return the random number and timestamp for round 100, which is a stale, already revealed value. This directly impacts downstream consumers such as `FlareSystemsManager`, which uses `getRandomNumber()` to select the vote power block.

Recommendations

Reject `bytes32(0)` Merkle roots in `relay` to eliminate the sentinel collision.

Remediation

This issue has been acknowledged by Flare Network.

Flare Network provided the following response to this finding:

The likelihood of a zero-value Merkle root is effectively zero. This aligns with the baseline cryptographic assumptions and risk tolerances currently standard across the blockchain sector.

3.11. Silent overflow in mulDivRoundUp

Target	SafePct		
Category	Coding Mistakes	Severity	Low
Likelihood	Low	Impact	Low

Description

The function `SafePct::mulDivRoundUp` computes $\text{ceil}(x * y / z)$ by first calling `mulDiv` for the floor, then conditionally adding 1 inside an unchecked block:

```
function mulDivRoundUp(uint256 x, uint256 y, uint256 z)
    internal pure returns (uint256) {
        uint256 resultRoundDown = mulDiv(x, y, z);
        unchecked {
            // safe - if z == 0, above mulDiv call would revert
            uint256 remainder = mulmod(x, y, z);
            // safe - overflow only possible if z == 1, but then remainder == 0
            return remainder == 0 ? resultRoundDown : resultRoundDown + 1;
        }
    }
}
```

The in-line comment claims the +1 can only overflow when $z == 1$, in which case $\text{remainder} == 0$ avoids the addition. This reasoning is incomplete. For any $z > 1$, it is possible to choose x and y such that $\text{floor}(x * y / z) == \text{type}(\text{uint256}).\text{max}$ while $\text{mulmod}(x, y, z) != 0$. In that case, the true ceiling is 2^{256} , which does not fit in a `uint256`, and the unchecked addition wraps to 0.

Here are the proof-of-concept inputs that trigger this behavior:

```
uint256 x =
115780511186197575666004384570230884764793505315109053134144169590954034236512;
uint256 y = 10001;
uint256 z = 10000;
// mulDiv(x, y, z) == type(uint256).max
// mulmod(x, y, z) == 6512
// mulDivRoundUp(x, y, z) == 0 (not revert)
```

Impact

There is no direct security impact of this issue as the callers in the codebase operate on values far below `type(uint256).max`. However, as a general-purpose utility library, future callers with larger value ranges could silently receive zero instead of a revert, leading to unexpected and potentially dangerous behavior.

Recommendations

Remove `unchecked` so that Solidity's default overflow checking catches the edge case.

Note that the same issue also exists in `SafePct : mulDivRoundUp` in the FSP V1 codebase. Because that version targets Solidity <0.8.0 (which lacks built-in overflow checks), the fix there should use `SafeMath.add` for the rounding increment instead to ensure overflow reverts.

Remediation

This issue has been acknowledged by Flare Network, and fixes were implemented in the following commits:

- [91f0f2fa](#) ↗
- [0b1f03cf](#) ↗

3.12. Missing initial checkpoint for partially zero public keys

Target	PublicKeyHistory		
Category	Coding Mistakes	Severity	Low
Likelihood	Low	Impact	Low

Description

In `PublicKeyHistory::setPublicKey`, when the checkpoint history is empty (`historyCount == 0`), a new checkpoint is created only if *both* key parts are nonzero:

```
if (_part1 != bytes32(0) && _part2 != bytes32(0)) {
    _self.checkpoints[0] = CheckPoint(_part1, _part2,
        SafeCast.toUint64(block.number));
    _self.endIndex = 1;
}
```

This condition is inconsistent with `EntityManager::registerPublicKey`, which accepts any key where *at least one* part is nonzero:

```
require(_part1 != bytes32(0) || _part2 != bytes32(0), "public key invalid");
```

As a result, if `registerPublicKey` passes a key with exactly one zero part while the history is empty, `setPublicKey` writes no checkpoint and returns silently. Then `registerPublicKey` continues execution, emits `PublicKeyRegistered`, and sets `publicKeyRegistered[hash] = msg.sender`.

This can create inconsistent state: 1) `publicKeyAtNow()` returns `(0, 0)` because no checkpoint was recorded; 2) `publicKeyRegistered[hash]` points to the voter, so the key appears registered; and 3) `unregisterPublicKey()` reads `(0, 0)` and returns early, leaving the mapping entry uncleared.

Impact

As of the time of writing, the `FastUpdater` verifier prevents this path from being reached; `verifyPublicKey` calls `Bn256.isG1PointOnCurve(pk)`, and no valid BN256 point has either coordinate equal to zero.

However, `publicKeyVerifier` is governance configurable via `setPublicKeyVerifier`. If a future verifier permits keys with a partially zero coordinate, a voter's `registerPublicKey` call could succeed without the key actually being recorded in the checkpoint history.

Recommendations

Change `&&` to `||` in the empty-history branch of `setPublicKey` to match the validation semantics in `registerPublicKey`:

```
if (_part1 != bytes32(0) && _part2 != bytes32(0)) {  
if (_part1 != bytes32(0) || _part2 != bytes32(0)) {
```

Remediation

This issue has been acknowledged by Flare Network.

Flare Network provided the following response to this finding:

The reported risk does not apply. We use the BN256 curve, which has no affine points with $x=0$ or $y=0$. Any valid on-curve point therefore has both coordinates non-zero.

3.13. The checkExecutorAndAllowedRecipient function treats address(0) as a valid delegation account when none exists

Target	ClaimSetupManager		
Category	Coding Mistakes	Severity	Low
Likelihood	Low	Impact	Low

Description

The internal helper `_getDelegationAccount` in `ClaimSetupManager` simply returns the stored delegation account address from the mapping, which defaults to `address(0)` when no delegation account has been created:

```
function _getDelegationAccount(address _owner)
    internal view returns (IIDelegationAccount) {
        return ownerToDelegationAccountData[_owner].delegationAccount;
    }
```

In `checkExecutorAndAllowedRecipient`, `_getDelegationAccount` is called twice without checking whether the returned address is nonzero:

```
function checkExecutorAndAllowedRecipient(address _executor,
    address _claimFor, address _recipient)
    external view override
{
    // checks if _executor is claiming for his account or his PDA account -
    // allow any _recipient
    if (_claimFor == _executor || _claimFor == address(_getDelegationAccount(
        _executor))) {
        return;
    }
    // if claiming for PDA, use owner settings
    address owner = delegationAccountToOwner[_claimFor];
    if (owner != address(0)) {
        _claimFor = owner;
    }
    // checks if _executor is allowed executor
    require(ownerClaimExecutorSet[_claimFor].index[_executor] != 0,
        ERR_ONLY_OWNER_OR_EXECUTOR);
    // checks if _recipient is allowed recipient
```

```
require(_recipient == _claimFor ||
    ownerAllowedClaimRecipientSet[_claimFor].index[_recipient] != 0 ||
    _recipient == address(_getDelegationAccount(_claimFor)),
    ERR_RECIPIENT_NOT_ALLOWED);
}
```

1. If the executor has no delegation account, `address(_getDelegationAccount(_executor))` evaluates to `address(0)`. So when `_claimFor == address(0)`, the function returns early and permits *any* recipient — bypassing the executor and recipient authorization checks entirely.
2. If `_claimFor` has no delegation account, `address(_getDelegationAccount(_claimFor))` evaluates to `address(0)`. This means `_recipient == address(0)` satisfies the recipient allowance check, even though no delegation account exists and `address(0)` is not an intentionally allowed recipient.

Impact

In the current codebase, as of the time of writing, the impact is mitigated because the callers reject `address(0)` recipients, and claims for `address(0)` do not produce meaningful rewards in normal operation.

However, the authorization helper is still logically incorrect and unsafe. Any future integration that relies on this function without independent `address(0)` guards can inherit unintended authorization behavior.

Recommendations

Guard delegation-account comparisons with explicit nonzero existence checks:

```
function checkExecutorAndAllowedRecipient(address _executor,
    address _claimFor, address _recipient)
    external view override
{
    // checks if _executor is claiming for his account or his PDA account -
    // allow any _recipient
    IDelegationAccount executorDelegationAccount = _getDelegationAccount(_
        executor);
    if (_claimFor == _executor || _claimFor == address(_getDelegationAccount(
        _executor))) {
        if (_claimFor == _executor ||
            (address(executorDelegationAccount) != address(0) && _claimFor ==
                address(executorDelegationAccount)))
```

```
{
    return;
}
// [...]
IIDelegationAccount ownerDelegationAccount = _getDelegationAccount(_
    claimFor);
require(_recipient == _claimFor ||
    ownerAllowedClaimRecipientSet[_claimFor].index[_recipient] != 0 ||
    _recipient == address(_getDelegationAccount(_claimFor)),
    (address(ownerDelegationAccount) != address(0) && _recipient ==
        address(ownerDelegationAccount)),
    ERR_RECIPIENT_NOT_ALLOWED);
}
```

Remediation

This issue has been acknowledged by Flare Network.

Flare Network provided the following response to this finding:

We have determined that no fix is required. The identified scenario is handled by existing validation logic within the delegation flow. As ClaimSetupManager is an immutable contract, we are accepting the current architecture as a permanent design choice.

3.14. Missing PDA status check allows claims to disabled PDA

Target	ClaimSetupManager		
Category	Coding Mistakes	Severity	Low
Likelihood	Low	Impact	Low

Description

The ClaimSetupManager manages personal delegation accounts (PDAs) for owners, and each PDA has an enabled flag stored in DelegationAccountData. When a user calls `disableDelegationAccount()`, `enabled` is set to `false`, signalling that rewards should flow to the owner's externally owned account (EOA) directly rather than to the PDA. The `getAutoClaimAddressesAndExecutorFee()` correctly respects this flag — it only returns the PDA address as the recipient when `delegationAccountData.enabled == true`; otherwise, it falls back to the owner address.

However, `checkExecutorAndAllowedRecipient()` — which is the gate enforced during actual executor-initiated claims in `FtsoRewardManager`, `DistributionToDelegators`, and `Escrow` — does not check the enabled flag at all when deciding whether the PDA is a valid recipient. It simply returns the stored delegation account address regardless of whether it is enabled. As a result, any executor can pass the owner's disabled PDA address as `_recipient` and the check will succeed, even though the owner has explicitly disabled it.

Impact

An executor can send claimed rewards to an owner's disabled PDA instead of the owner's own address, directly violating the owner's express configuration. Note that the severity is marked as Low because users can still delegate, undelegate, or withdraw funds. Additionally, the owner can reenable the PDA to use the full functionality.

Recommendations

Add an enabled check inside `checkExecutorAndAllowedRecipient()` before permitting the PDA as a valid recipient. Mirror the same logic used in `getAutoClaimAddressesAndExecutorFee()`. This change ensures that a disabled PDA is never accepted as a valid claim destination during executor-initiated claims, making the two code paths consistent and fully respecting the owner's express configuration.

```
function checkExecutorAndAllowedRecipient(address _executor,
    address _claimFor, address _recipient)
```



```
external view override
{
    [...]
    // checks if _executor is allowed executor
    require(ownerClaimExecutorSet[_claimFor].index[_executor] != 0,
    ERR_ONLY_OWNER_OR_EXECUTOR);
    // checks if _recipient is allowed recipient
    require(_recipient == _claimFor ||
        ownerAllowedClaimRecipientSet[_claimFor].index[_recipient] != 0 ||
        _recipient == address(_getDelegationAccount(_claimFor)),
        (ownerToDelegationAccountData[_claimFor].enabled &&
        _recipient == address(_getDelegationAccount(_claimFor))),
        ERR_RECIPIENT_NOT_ALLOWED);
}
```

Remediation

This issue has been acknowledged by Flare Network.

Flare Network provided the following response to this finding:

Acknowledged. As the contract is immutable and no funds are at risk, this is classified as a low-impact operational inconvenience and will not be remediated.

3.15. Nonstandard initialization pattern

Target	FdcVerification, FtsoV2		
Category	Coding Mistakes	Severity	Low
Likelihood	Low	Impact	Low

Description

The FdcVerification and FtsoV2 contracts inherit from UUPSUpgradeable but use a custom initialization mechanism (`GovernedBase::initialise()`) instead of OpenZeppelin's standard Initializable pattern (`initializer / reinitializer` modifiers). The `initialize()` function calls `GovernedBase.initialise()`, which uses a simple boolean flag (`initialised`) to prevent reexecution. This design prevents any reinitialization logic from running during contract upgrades.

Impact

Because the initialized flag is a single boolean with no version counter, there is no way to safely initialize new state variables introduced in future upgraded implementations. Any upgrade requiring post-upgrade initialization logic forces each upgrade to create new state variables to prevent reexecution.

Recommendations

Consider refactoring the initialization mechanism to use OpenZeppelin's standard initialization pattern and calling `_disableInitializers()` in the implementation constructor. This maintains backward compatibility while enabling proper reinitialization in future upgrades.

Remediation

This issue has been acknowledged by Flare Network.

Flare Network provided the following response to this finding:

Given a very limited impact, remediation is deemed unnecessary as the contract is permanently deployed.

3.16. The requestMinting function does not validate the cumulative pending amount against maxMintingRequestWei

Target	FlareDaemon		
Category	Code Maturity	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

In FlareDaemon::requestMinting, validation is applied only to the incoming request amount:

```
function requestMinting(uint256 _amountWei) external onlyInflation(msg.sender)
{
    require(_amountWei <= maxMintingRequestWei, ERR_TOO_BIG);
    require(_getNextMintRequestAllowedTs() < block.timestamp, ERR_TOO_OFTEN);
    if (_amountWei > 0) {
        lastMintRequestTs = block.timestamp;
        totalMintingRequestedWei = totalMintingRequestedWei.add(_amountWei);
        emit MintingRequestReceived(_amountWei);
    }
}
```

Meanwhile, the pending mint amount is computed as totalMintingRequestedWei - totalMintingReceivedWei:

```
function getPendingMintRequest()
private view returns(uint256 _mintRequestPendingWei) {
    _mintRequestPendingWei
    = totalMintingRequestedWei.sub(totalMintingReceivedWei);
}
```

Because requestMinting does not enforce getPendingMintRequest() + _amountWei <= maxMintingRequestWei, the request can accumulate into a pending total above the configured cap.

Impact

There is no practical impact under the integration as of the time of writing, because the Inflation contract already accounts for pending minting before calling requestMinting.

However, for defense in depth, FlareDaemon should enforce this invariant directly instead of

relying on external caller behavior.

Recommendations

Include the already pending amount in the validation:

```
function requestMinting(uint256 _amountWei) external onlyInflation(msg.sender)
{
    require(_amountWei <= maxMintingRequestWei, ERR_TOO_BIG);
    require(_amountWei.add(getPendingMintRequest()) <= maxMintingRequestWei,
        ERR_TOO_BIG);
    // [...]
}
```

Remediation

This issue has been acknowledged by Flare Network.

Flare Network provided the following response to this finding:

Acknowledged. Updating FlareDaemon would require a hard fork, so we have included the missing logic in the Inflation contract.

3.17. The 30/360 day-count convention causes effective calendar-year inflation to exceed stated annual rate

Target	InflationTimeSlots		
Category	Business Logic	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

The InflationTimeSlots library defines time slots as fixed 30-day periods and computes per-slot recognized inflation by dividing the annual rate by 12:

```
function _getTimeSlotEndsTs(uint256 _startTimeStamp)
    private pure returns (uint256) {
        // This should cover passing through Feb 29
        return _startTimeStamp.add(SECONDS_PER_DAY * 30).sub(1);
    }
```

```
function _computeRecognizedInflationWei(
    uint256 _inflatableBalance,
    uint256 _timeSlotInflationPercentageBips
)
    private pure
    returns(uint256)
{
    return Math.min(
        _inflatableBalance.mulDiv(_timeSlotInflationPercentageBips,
            12 * BIPS100),
        MAX_ANNUAL_INFLATION.div(12)
    ); // monthly inflation
}
```

This follows a 30/360 day-count convention common in traditional finance. Within a 360-day protocol year (12 slots), the math is internally consistent; the full intended annual rate is recognized, and the MAX_ANNUAL_INFLATION cap holds exactly.

However, a calendar year is 365 days (366 in a leap year), so approximately 12.17 slots execute per calendar year. Each slot recognizes $\text{annualRate} / 12$ of inflation, resulting in a cumulative 365-day recognized inflation of $\sim 12.17 / 12 \approx 1.0139x$ the stated annual rate.

Impact

The impact is limited to a documentation and perception concern. The effective 365-day inflation rate exceeds the nominal annual rate by ~1.39%. The constant name `MAX_ANNUAL_INFLATION` and the NatSpec phrase "per-annum basis" in `InflationAllocation` could be interpreted as calendar-year references, potentially causing confusion among token holders or external integrators reviewing the contracts.

Recommendations

Add documentation clarifying that the protocol uses a 30/360 day-count convention, that "annual" in the context of inflation parameters refers to a 360-day cycle, and that the effective calendar-year inflation rate is marginally higher than the stated rate (~1.39%).

Remediation

This issue has been acknowledged by Flare Network.

Flare Network provided the following response to this finding:

This is an intentional design choice. The current architecture was selected to reduce complexity.

3.18. Unhandled return value of `daemonize()`

Target	FlareDaemon		
Category	Coding Mistakes	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

The `IFlareDaemonize` interface declares `daemonize()` with a `bool` return value:

```
function daemonize() external returns (bool);
```

However, `FlareDaemon::triggerInternal` discards this return value in its `try` block. The success branch always emits `ContractDaemonized` regardless of what the callee returned:

```
try daemonizedContract.daemonize{gas: useGas}() {
    emit ContractDaemonized(address(daemonizedContract), (startGas
    - gasleft()));
```

Impact

Because the `bool` return value is discarded and not included in the `ContractDaemonized` event, off-chain indexers and monitoring systems have no practical way to determine whether a `daemonize()` call actually performed work or was a no-op.

Recommendations

Surface the return value in the event so off-chain consumers can differentiate:

```
event ContractDaemonized(address theContract, uint256 gasConsumed,
    bool workPerformed);

// ...
try daemonizedContract.daemonize{gas: useGas}() returns (bool workPerformed) {
    emit ContractDaemonized(address(daemonizedContract), (startGas
    - gasleft()), workPerformed);
```

Remediation

This issue has been acknowledged by Flare Network.

Flare Network provided the following response to this finding:

Acknowledged. This would require a hard fork to resolve; since there is no impact, the risk is accepted.

3.19. The clear function does not deduplicate same-block checkpoints

Target	DelegationHistory		
Category	Business Logic	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

In DelegationHistory, the writeValue function handles the case where the latest checkpoint's fromBlock matches the current block.number by updating the existing checkpoint in place:

```
if (block.number == lastBlock) {
    // If last check point is the current block, just update
    _updateDelegates(lastCheckpoint, _delegate, _value);
}
```

However, the clear function unconditionally appends a new empty checkpoint without checking for this same-block scenario:

```
function clear(CheckPointHistoryState storage _self) internal {
    uint256 historyCount = _self.length;
    if (historyCount > 0) {
        // add an empty checkpoint
        CheckPoint storage cp = _self.checkpoints[historyCount];
        _self.length = SafeCast.toUint64(historyCount + 1);
        // create empty checkpoint = only set fromBlock
        cp.delegations[0] = Delegation({
            delegate: address(0),
            value: 0,
            fromBlock: block.number.toUint64(),
            length: 0
        });
    }
}
```

When a user calls _delegateByPercentage (which calls delegation.addReplaceDelegate → DelegationHistory.writeValue) and subsequently calls _undelegateAllByPercentage (which calls delegation.clear → DelegationHistory.clear) within the same block, two checkpoints will be created with identical fromBlock values.

Impact

Correctness is not affected; the binary search always returns the rightmost index among duplicates, so the latest state is retrieved correctly. The impact is minor storage waste (one redundant checkpoint per occurrence) and a marginally increased cost during `cleanupOldCheckpoints`.

Recommendations

Add the same-block deduplication check to `clear`, matching the pattern already used in `writeValue`. When the latest checkpoint is from the current block, clear it in place instead of appending a new one.

Remediation

This issue has been acknowledged by Flare Network.

Flare Network provided the following response to this finding:

This is an intentional design choice, with no impact on security.

3.20. Zero-value transfer during initial VPToken setup can force replacement-mode VPCContract

Target	VPToken		
Category	Coding Mistakes	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

Regardless of the transfer amount, VPToken::_beforeTokenTransfer sets `vpContractInitialized = true` whenever a transfer occurs while `writeVpContract` is unset:

```
function _beforeTokenTransfer(
    address _from,
    address _to,
    uint256 _amount
)
    internal virtual
    override(ERC20)
{
    // [...]
    IIVPCContract vpc = writeVpContract;
    if (address(vpc) != address(0)) {
        vpc.updateAtTokenTransfer(_from, _to, fromBalance, toBalance,
            _amount);
    } else if (!vpContractInitialized) {
        // transfers without vpcontract are allowed, but after they are made
        // any added vpcontract must have isReplacement set
        vpContractInitialized = true;
    }
    // [...]
}
```

Because OpenZeppelin's ERC20 permits zero-value transfers, any EOA (even one with no token balance) can trigger this state transition by calling `transfer(someAddress, 0)` before governance performs initial VPCContract setup.

Once `vpContractInitialized` is true, `setWriteVpContract` rejects any VPCContract that is not configured for replacement:

```
function setWriteVpContract(IIVPContract _vpContract)
    external onlyGovernance {
        if (address(_vpContract) != address(0)) {
            // [...]
            require(!vpContractInitialized || _vpContract.isReplacement(),
                "VPContract not configured for replacement");
            // [...]
        }
        // [...]
    }
}
```

There is no path to reset `vpContractInitialized` to `false`. Thus, during the deployment window (after token deployment, before first `setWriteVpContract`), an attacker can front-run governance's setup transaction with a zero-value transfer, causing the nonreplacement `VPContract` configuration to revert and forcing governance to deploy a replacement-mode `VPContract` instead.

Impact

As of the time of writing, the currently deployed `WNat` contract is unaffected, as its `VPContract` is already configured. This issue only applies to future deployments of `VPToken`-derived contracts, during the one-time window between deployment and the first `setWriteVpContract` call. An attacker exploiting this window can prevent governance from configuring a nonreplacement `VPContract`.

Recommendations

Gate initialization on nonzero token movement:

```
} else if (!vpContractInitialized && _amount > 0) {
    vpContractInitialized = true;
}
```

Alternatively, governance can avoid the issue operationally by deploying the token and calling `setWriteVpContract` atomically via a deployer contract.

Remediation

This issue has been acknowledged by Flare Network.

Flare Network provided the following response to this finding:

Acknowledged. In the highly improbable event of a future contract upgrade, this will be strictly monitored during the migration. Otherwise, the risk remains zero as there is no viable exploit path.

4. Discussion

The purpose of this section is to document miscellaneous observations that we made during the assessment. These discussion notes are not necessarily security related and do not convey that we are suggesting a code change.

4.1. Relay migration risk in RandomNumberV2 integrations

After a new Relay is deployed, the old Relay may stop receiving random-number updates. At that point, any consumer still pointing to the old RandomNumberV2 address will stop receiving fresh randomness.

The current Developer Hub example, "[Use secure random onchain](#)", reads the random number provider once in the constructor and stores it in an immutable variable:

```
contract SecureRandomConsumer {
    /// @notice Random number provider (fetched from ContractRegistry at
    /// construction).
    RandomNumberV2Interface public immutable randomV2;

    /**
     * @notice Initialize and grab the RandomNumberV2 instance from the
     * ContractRegistry.
     */
    constructor() {
        randomV2 = ContractRegistry.getRandomNumberV2();
    }
}
```

This pattern is not safe across Relay migrations. If a new Relay/RandomNumberV2 contract is deployed and the old Relay is no longer updated, any contract that followed this example will remain permanently pinned to the deprecated instance and will no longer be able to read fresh randomness from the active Relay.

We recommend documenting explicitly that the RandomNumberV2 address may change over time and should not be treated as immutable. Instead, integrators should resolve the current address dynamically via `ContractRegistry.getRandomNumberV2()` when reading randomness.

Flare Network provided the following response to this finding:

Fixed in Flare Developer Hub: <https://dev.flare.network/network/guides/secure-random-numbers#use-secure-random-onchain>

5. System Design

This provides a description of the high-level components of the system and how they interact, including details like a function's externally controllable inputs and how an attacker could leverage each input to cause harm or which invariants or constraints of the system are critical and must always be upheld.

Not all components in the audit scope may have been modeled. The absence of a component in this section does not necessarily suggest that it is safe.

5.1. Governance

Description

The Governance system provides time-locked administrative control over FSP protocol parameters and upgrades. It implements a two-phase governance model with an initial setup phase and production mode with mandatory timelocks. Shown below is the core functionality of the Governance contract:

1. **Timelock management.** It queues and executes governance calls with mandatory delays.
2. **Production mode.** It switches from initial governance to production governance with timelocks.
3. **Call initiation.** It initiates timelocked governance calls via governance.
4. **Call execution.** It executes previously timelocked governance calls via authorized executor.
5. **Emergency controls.** It cancels pending timelocked calls when necessary.
6. **Access control.** It separates governance and executor roles for security.

Invariants

The following invariants must hold true:

1. Timelocked calls require proper timelock expiration before execution.
2. Only governance can initiate timelock calls via the `onlyGovernance` modifier.
3. Only executors can execute timelocked calls via the `isExecutor()` check.
4. Only governance can execute immediately calls via the `onlyImmediateGovernance` modifier.
5. Production mode cannot be reverted once activated.
6. Immediate governance calls by the initial governance require nonproduction mode.

7. Only one pending timelocked call per function selector can exist at a time.
8. Executor authorization must be validated through governance settings.
9. Timelock delay relies on governance settings at the moment initiate calls are made.

Attack surface

The following outlines the attack surface:

1. **Timelock bypass.** This is prevented via mandatory timelock enforcement in production mode.
2. **Unauthorized execution.** This is prevented via executor-authorization checks.
3. **Call replay.** This is prevented via function-selector tracking and deletion after execution.
4. **Production-mode bypass.** This is prevented via irreversible production-mode switch.
5. **Emergency response.** Governance can cancel pending calls to respond to threats.

5.2. Inflation

Description

Inflation is a sophisticated token emission system that manages the minting and distribution of native tokens to reward service contracts on the Flare network. It operates on a monthly time-slot basis where each slot recognizes a predetermined amount of inflation (calculated as a percentage of the inflatable supply), then distributes this amount daily over 30 days to multiple reward receivers (like FTSO, FDC, validators, etc.) according to governance-defined sharing percentages. The system implements a two-phase top-up mechanism:

1. Daily authorization calculates how much each receiver is entitled to receive.
2. Top-up computation determines how much native token to mint based on configurable reserve strategies.

Shown below is the core functionality of Inflation:

1. **Time-slot management.** It initializes monthly inflation periods with recognized inflation amounts based on inflatable balance and percentage schedule.
2. **Daily authorization.** It distributes recognized inflation proportionally across reward services over remaining days in the time slot.
3. **Top-up calculation.** It computes minting requests to maintain claim reserves in reward contracts using FACTOROFDAILYAUTHORIZED.

4. **Minting distribution.** It receives minted tokens from FlareDaemon and transfers them to reward service contracts.
5. **Governance controls.** It allows updating sharing percentages, top-up configurations, and inflation schedules through the InflationAllocation contract.

Invariants

The following invariants must hold true:

1. It cannot authorize more inflation than has been recognized for the time slot via the `totalAuthorizedInflationWei <= totalRecognizedInflationWei` check.
2. It cannot request minting more than authorized via the `totalInflationTopupRequestedWei <= totalAuthorizedInflationWei` check.
3. It cannot distribute more than requested via the `totalInflationTopupDistributedWei <= totalInflationTopupRequestedWei` check.
4. The sum of all receiver percentages must equal exactly 10,000 bips (100%).
5. New time slots must start after the previous slot ends.
6. Inflation authorization occurs exactly once per day (86,400 seconds).
7. Inflation's pending requests must account for daemon's pending via the `pendingMintRequestInflation >= pendingMintRequestFlareDaemon` check.

Attack surface

The following outlines the attack surface:

1. **Arithmetic overflow.** Most arithmetic operations use OpenZeppelin's SafeMath library, except in `setInitialData()`; see Finding [3.6](#).
2. **Self-destruct fund injection.** This is prevented via `receiveMinting()`; any balance exceeding the expected top-up amount is detected and immediately burned to `BURN_ADDRESS`, preventing fund accumulation.
3. **Top-up request manipulation.** Malicious reward receivers could manipulate `getExpectedBalance()` to request excessive top-ups. However, this is prevented because top-up calculations are capped by `maxTopupRequestWei = authorizedInflationWei - pendingTopup - distributedWei`. If total requests exceed `maxMintingRequestWei`, they are proportionally reduced.

5.3. FlareDaemon

Description

The FlareDaemon contract is a system-level contract that orchestrates periodic execution of registered contracts through a validator-triggered polling mechanism. It serves as the heartbeat of the Flare network, coordinating two primary functions:

1. **Daemonization service.** It systematically calls `daemonize()` on registered contracts each block to enable time-based operations like FTSO price delta application, inflation distribution, and reward calculations.
2. **Native-token minting coordinator.** It manages inflation requests from the Inflation contract to the validator, handling minted token receipts and forwarding funds while tracking self-destruct proceeds.

Shown below is the core functionality of the FlareDaemon contract:

1. **Contract registration and polling.** Governance registers up to 10 contracts implementing `IFlareDaemonize` with individual gas limits; the daemon calls each contract's `daemonize()` method sequentially per block in registration order.
2. **Minting request management.** It accepts minting requests from the Inflation contract (maximum 60M tokens by default, maximum once per 23 hours), queues them, and returns the amount to the validator via the `trigger()` return value.
3. **Minting receipt and distribution.** It receives minted tokens from the validator, distinguishes between expected minting vs self-destruct proceeds through balance accounting, and forwards minted amounts to the Inflation contract.
4. **Error handling and fallback.** It catches and logs all `daemonize` errors by type (contract plus message hash), attempts to switch contracts to fallback mode on failure, and implements hold-off periods for out-of-gas contracts.
5. **Gas management.** It enforces per-contract gas limits, reserves minimum gas between calls, skips remaining contracts when insufficient gas remains, and tracks gas consumption per call.

Invariants

The following invariants must hold true:

1. The `trigger()` method can only execute once per block (enforced by the `systemLastTriggeredAt == block.number` check), providing implicit reentrancy protection.

5. **Balance manipulation via self-destruct.** The contract detects unexpected balance increases by comparing `currentBalance` against `lastBalance + expectedMintRequest`, categorizing excess funds as self-destruct proceeds and tracking them separately in `totalSelfDestructReceivedWei` without disrupting normal minting operations.
6. **Cascading contract failures.** This is prevented via try-catch blocks around each `daemonize()` call that isolate errors, automatic fallback mode switching via `switchToFallbackMode()`, and continuation of processing remaining contracts even when one fails.

5.4. ClaimSetupManager

Description

The ClaimSetupManager contract is a sophisticated reward-claiming orchestration system that manages automatic reward claiming through registered executors and personal delegation accounts (PDAs). The contract enables users to delegate their claiming authority to executors who can claim FTSO and airdrop rewards on their behalf, while PDAs serve as separate contracts that hold wrapped native tokens (WNat) and maintain independent delegation settings, allowing users to separate their voting power delegation from their main account while still controlling the funds. Shown below is the core functionality of the ClaimSetupManager contract:

1. **Executor registration and fee management.** Executors register by paying a fee (burned to `BURN_ADDRESS`) and set their service fees within governance-defined bounds, with fee updates subject to a timelock (`feeValueUpdateOffset`) to prevent sudden changes and protect users from fee manipulation.
2. **PDA management.** Users can create minimal proxy clones of a library implementation to establish PDAs that receive autoclaimed rewards, enabling separation of voting power delegation from main account holdings while maintaining full control over funds.
3. **Claim authorization and recipient control.** Users configure authorized executors and allowed recipient addresses, with the system enforcing that executors can only claim to approved destinations (owner, PDA, or explicitly whitelisted addresses) to prevent fund theft.
4. **Delegation operations proxy.** It provides a convenient interface for users to manage WNat vote power delegation and governance delegation through their PDAs without directly interacting with the PDA contract.
5. **Executor discovery.** It maintains a registry of executors with their fee schedules and provides batch query functions for reward managers to efficiently retrieve claiming addresses and executor fees.

Invariants

The following invariants must hold true:

1. Executor-fee changes are always scheduled at least `feeValueUpdateOffset` epochs in the future, ensuring users have advance notice before fee increases take effect.
2. All executor fees remain within the governance-controlled bounds `[minFeeValueWei, maxFeeValueWei]` at all times, preventing excessive fees.
3. Each user can have at most one PDA, and once created, the PDA-to-owner mapping is immutable, establishing a permanent one-to-one relationship.
4. The `WNat` address can only be set once during initialization and cannot be changed afterward, ensuring consistent token handling.
5. Only the `ClaimSetupManager` (acting as manager) can invoke PDA functions, preventing unauthorized manipulation of user funds.
6. Executors must be explicitly authorized by the owner in `ownerClaimExecutorSet` before they can claim on behalf of that owner.
7. When setting new executors, users must pay the current fee to any newly added registered executors, ensuring executors are compensated up front.
8. The owner always has implicit permission to claim to their own address or their PDA regardless of allowed recipient settings.
9. Disabled PDAs cannot receive autoclaimed rewards, with the system falling back to the owner's address.
10. Executor registration fees are immediately and permanently burned, making registration irreversible from a fee perspective.

Attack surface

The following outlines the attack surface:

1. **Reentrancy during executor-fee payment.** This is prevented via the `nonReentrant` modifier on `setAutoClaiming` and `setClaimExecutors`, protecting against malicious executors attempting to reenter during the fee payment callback.
2. **Executor claiming to arbitrary recipients.** This is prevented via `checkExecutorAndAllowedRecipient`, which validates that recipients are either the owner, the owner's PDA, or explicitly whitelisted in `ownerAllowedClaimRecipientSet`.
3. **Front-running executor fee increases.** This is prevented via the timelock mechanism where fee updates only become effective after `feeValueUpdateOffset` epochs, giving users time to remove executors before higher fees apply.
4. **Malicious executor draining user funds through excessive fees.** This is prevented via the up-front fee payment model where users pay executor fees when adding executors, and the fee is capped by a governance-set maximum, limiting potential losses.

5. **PDA impersonation or spoofing.** This is prevented via the immutable `delegationAccountToOwner` mapping established during PDA creation and via validation in `checkExecutorAndAllowedRecipient` that resolves PDAs to their owners.
6. **Excess payment theft during executor setup.** This is prevented via explicit refund logic in `_setClaimExecutors` that returns `msg.value - totalExecutorsFee` to the user after paying required executor fees.
7. **Historical fee manipulation.** This is prevented via the assertion and validation in `_updateExecutorFeeValue` that ensures `_validFromEpoch >= lastValidFromEpoch`, preventing updates to past epochs.

5.5. Supply

Description

The Supply contract serves as the central accounting system for tracking native-token supply metrics across the Flare network, maintaining historical checkpointed records of circulating supply and computing the inflatable balance by aggregating data from token pools, burn addresses, and foundation holdings. Shown below is the core functionality of the Supply contract:

1. **Circulating-supply tracking.** It maintains checkpointed history of circulating supply that accounts for claims, locks, burns, and foundation balance changes.
2. **Inflatable balance calculation.** It computes the total supply available for inflation by adding the genesis amount and claimed rewards then subtracting excluded/locked amounts.
3. **Token-pool integration.** It registers token pools and aggregates their supply data (locked funds, authorized inflation, claimed amounts) during updates.
4. **Foundation-address management.** It tracks designated foundation addresses and excludes their balances from the circulating supply.
5. **Burn-address accounting.** It monitors two burn addresses and removes their balances from the circulating supply.
6. **Governance controls.** It allows governance to add token pools, adjust the distributed supply, and modify foundation addresses.

Invariants

The following invariants must hold true:

1. The distributed excluded supply cannot exceed the total excluded supply (`distributedExcludedSupplyWei <= totalExcludedSupplyWei`).

2. The sum of locked funds and authorized inflation from each token pool must always be greater than or equal to the claimed funds from that pool (`newTotalLockedWei + newTotalInflationAuthorizedWei >= newTotalClaimedWei`).
3. Burn-address balances can only increase over time, monotonically decreasing circulating supply.
4. Burn-address balances are permanently excluded from the circulating supply and can only increase.
5. Only the Inflation contract can trigger supply updates, ensuring synchronized state transitions.
6. Each token pool can only be registered once in the system.
7. The initial genesis amount must be greater than zero to prevent invalid supply initialization.

Attack surface

The following outlines the attack surface:

1. **Unauthorized supply manipulation.** This is prevented via the `onlyInflation` modifier on `updateCirculatingSupply` and `updateAuthorizedInflationAndCirculatingSupply`, restricting supply updates to only the Inflation contract.
2. **Excessive distributed supply.** This is prevented via the assertion `totalExcludedSupplyWei.sub(distributedExcludedSupplyWei) >= _amountWei` in `_increaseDistributedSupply`, ensuring distributed amounts never exceed excluded supply.
3. **Invalid token pool state.** This is prevented via the assertion `newTotalLockedWei.add(newTotalInflationAuthorizedWei) >= newTotalClaimedWei` in `_updateCirculatingSupply`, ensuring claimed amounts are always backed by locked or authorized funds.
4. **Token-pool double registration.** This is prevented via the `_addTokenPool` function checking if a token pool already exists in the array before adding it.

5.6. Fast updates

Description

Fast updates is a high-frequency oracle system that enables authorized providers to submit rapid price-feed updates through a sortition-based selection mechanism using verifiable random function (VRF) proofs. Shown below is the core functionality of Fast updates:

1. **Feed submission and validation.** Providers submit delta-encoded updates with sortition credentials proving eligibility, verified via ECDSA signatures and VRF proofs on the BN256 elliptic curve.
2. **Sortition-based provider selection.** Each block initiates a new sortition round where providers with weight-proportional probability can submit updates if their VRF output score falls below a dynamically computed cutoff threshold.
3. **Delta encoding and backlog management.** Updates are stored as two-bit delta values (multiply/divide by scale, or no change) in a circular buffer of 500 entries, applied lazily to feed values during daemon calls or fetch operations.
4. **Dynamic scale adjustment.** Feed values maintain 32-bit precision with adjustable decimal places, automatically rescaled when values grow too large or become too small to benefit from scale multiplication.
5. **Incentive-driven performance tuning.** External parties can pay to temporarily increase the range (volatility tolerance) and sample size (update frequency) parameters, with costs calculated via exponential pricing for sample size increases and linear pricing for range increases.
6. **Fee-based data access.** Consumers pay configurable fees (per-feed, per-category, or default) to fetch current feed values, with governance-controlled free access for whitelisted addresses.
7. **Configuration management.** Governance controls feed addition/removal, reward band values, and inflation share distribution through the FastUpdatesConfiguration contract.

Invariants

The following invariants must hold true:

1. The submission window must be less than MAX_BLOCKS_HISTORY (100 blocks) to ensure score cutoff thresholds remain accessible for validation.
2. Feed values are constrained to 32 bits with the upper three bits always zero, ensuring values remain below 2^{29} to prevent overflow during scale multiplication.
3. The precision (range/sample size) must never exceed 100% and must be at least 2^{-25} to ensure meaningful update granularity.
4. Each provider-replicate combination can submit at most once per sortition block, enforced by storing hashed randomness values in submittedHashes.
5. The replicate value in sortition credentials must be strictly less than the provider's weight, ensuring valid replicate indexing.
6. The circular delta backlog maintains ordering where backlogDelta tracks the oldest unprocessed update and currentDelta tracks the next write position, with updates applied sequentially during _applySubmitted.

7. VRF gamma points and provider public keys must lie on the BN256 curve, verified via the equation $y^2 = x^3 + 3 \bmod p$.
8. The range increase limit must be greater than or equal to the base range, and the range must always be less than the sample size.
9. Feed decimals are stored as signed eight-bit integers, allowing values from -128 to 127 to accommodate extreme scaling scenarios.
10. Score cutoffs are precomputed per block and stored in a circular buffer of length `submissionWindow + 1`, ensuring $O(1)$ lookup during submission validation.
11. Delta values encode three operations: 00 (no change), 01 (multiply by scale), and 11 (divide by scale), with 10 reserved/unused.
12. The `currentScale` is updated by the daemon each block based on the current precision from the incentive manager, while feed-specific decimals use `baseScale` during reset operations.

Attack surface

The following outlines the attack surface:

1. **Replay attacks across sortition rounds.** This is prevented via the `submittedHashes` mapping that stores `sha256(pubKey, sortitionBlock, replicate)` for each submission, rejecting duplicate submissions within the submission window.
2. **Unauthorized feed updates from nonproviders.** This is prevented via ECDSA signature verification where the recovered `signingPolicyAddress` must be registered in the `VoterRegistry` with nonzero public key and normalized weight. However, a provider might submit unauthorized feed updates at epoch boundaries due to a sortition-block parameter mismatch issue; see Finding [3.2](#).
3. **Delta backlog overflow.** This is prevented via the fixed-size circular buffer of `MAX_SUBMITTED_DELTAS_BACKLOG` (500) entries with modulo arithmetic wrapping, though **note** that rapid submissions could theoretically overwrite unprocessed deltas if `currentDelta` catches up to `backlogDelta`.
4. **Precision manipulation through incentive offers.** This is prevented via governance-controlled `sampleIncreaseLimit` and `rangeIncreaseLimit` parameters, with additional validation that precision cannot exceed 100% and the range increase price must make $1e-6$ changes cost at least 1 wei.
5. **Fee bypass for data access.** This is prevented via the `FeeCalculator` contract computing required fees based on feed-specific, category-specific, or default rates, with payment validation in `fetchCurrentFeeds` unless the caller is in the `freeFetchAddresses` whitelist.

6. **Integer overflow in fixed-point arithmetic.** This is prevented via explicit `check()` functions that validate all `Scale`, `SampleSize`, `Range`, and `Fee` values remain below 2^{128} after operations.
7. **Incentive-offer value overflow.** This is prevented via the requirement `msg.value >> 120 == 0`, capping incentive offers at 120 bits to prevent arithmetic issues in fixed-point calculations.

5.7. Flare Time Series Oracle (FTSO)

Description

Flare Time Series Oracle (FTSO) is a decentralized oracle system that publishes price-feed data on chain through a cryptographically verified mechanism and manages reward distribution for data providers. The system operates through voting rounds where data providers submit price data, which is aggregated and published with Merkle proof verification, while rewards are distributed through both inflation-based and community-offered mechanisms with configurable parameters for feed-specific reward bands and turnout thresholds. Shown below is the core functionality of the FTSO:

1. **Feed publishing.** It accepts price-feed data with Merkle proofs verified against relay-stored Merkle roots.
2. **Reward offers management.** It handles both community-offered rewards and inflation-based rewards, distributing funds to the `RewardManager` contract based on configurable FTSO configurations with primary/secondary reward bands and minimum turnout requirements.

Invariants

The following invariants must hold true:

1. Published feeds must have voting round IDs strictly less than the current voting round ID obtained from the relay contract.
2. The sum of all reward offer amounts in `offerRewards` must exactly equal the `msg.value` sent with the transaction.
3. Reward offers can only be made for the next reward epoch (current plus 1) and must be submitted before the signing-policy initialization window begins.
4. Decimal changes scheduled by governance become effective only after `decimalsUpdateOffset` reward epochs, and past settings cannot be modified.
5. Total inflation rewards offered cannot exceed total inflation received, maintaining the invariant `totalInflationRewardsOfferedWei <= totalInflationReceivedWei`.

6. Inflation reward distribution proportionally allocates rewards based on each configuration's `inflationShare` relative to the total share sum, with remaining amounts adjusted iteratively to prevent rounding errors.
7. Feed IDs are exactly 21 bytes consisting of a one-byte category followed by up to 20 bytes for the name.
8. Reward band parameters (`minRewardedTurnoutBIPS`, `primaryBandRewardSharePPM`, `secondaryBandWidthPPM`) must not exceed their respective maximums (10,000 bips, 1,000,000 PPM).

Attack surface

The following outlines the attack surface:

1. **Merkle-proof forgery.** This is prevented via cryptographic verification in `publish()` where `proof.merkleProof.verifyCalldata(merkleRoot, feedHash)` validates each feed against the Merkle root stored in the Relay contract for the specific protocol ID and voting round.
2. **Reward amount mismatch.** This is prevented via the exact equality check `require(sumRewardsOfferValues == msg.value)` ensuring no funds are lost or stolen during reward-offer submission.
3. **Unauthorized feed publishing.** This is prevented via the `onlyFeedsPublisher` modifier on `publishFeeds()` restricting privileged publishing to the governance-set feeds publisher address, while public `publish()` requires Merkle proof verification.
4. **Decimal manipulation.** This is prevented via the `decimalsUpdateOffset` delay mechanism and the assertion `assert(rewardEpochId >= decimalsForFeedId[position - 1].validFromEpochId)` preventing retroactive decimal changes that could affect past reward calculations.
5. **Stale feed access.** This is prevented via the `require(minVotingRoundId <= _votingRoundId)` check in `getFeed()` and the additional verification `require(_feed.votingRoundId == _votingRoundId)` ensuring the retrieved feed matches the requested voting round.
6. **Inflation reward calculation manipulation.** This is prevented via the time-weighted calculation `totalRewardsAmount = (totalInflationReceivedWei - totalInflationRewardsOfferedWei).mulDiv(_rewardEpochDurationSeconds, intervalEnd - intervalStart)` and iterative share-based distribution that maintains mathematical consistency.

5.8. Flare Data Connector (FDC)

Description

Flare Data Connector (FDC) is a decentralized attestation system that enables external data verification on the Flare network through cryptographic proofs. The system consists of three core components:

1. **FdcHub.** This manages attestation requests and distributes inflation-based rewards to data providers.
2. **FdcVerification.** This validates Merkle-proof-based attestations against finalized voting round data from the Relay contract.
3. **FdcInflationConfigurations and FdcRequestFeeConfigurations.** These govern the economic parameters for different attestation types and data sources.

Shown below is the core functionality of the FDC:

1. **Attestation-request processing.** It allows users to submit requests with fees that are routed to reward pools for specific reward epochs.
2. **Verification system.** It supports seven attestation types (AddressValidity, BalanceDecreasingTransaction, ConfirmedBlockHeightExists, EVMTransaction, Payment, ReferencedPaymentNonexistence, Web2Json) where each proof contains a Merkle path and response data validated against stored Merkle roots.
3. **Inflation distribution.** It uses a time-weighted allocation mechanism that distributes accumulated inflation proportionally across reward epochs based on epoch duration and inflation receipt timing.
4. **Fee configuration.** It maps attestation type-source pairs to required fees, ensuring only supported attestation types can be requested and properly incentivized.
5. **Upgradability.** It is implemented via UUPS proxy pattern for FdcVerification, allowing governance to upgrade verification logic while maintaining state continuity.

Invariants

The following invariants must hold true:

1. The balance of FdcHub must always equal `totalInflationReceivedWei - totalInflationRewardsOfferedWei`, enforced by the `mustBalance` modifier that burns any excess funds from self-destruct attacks.
2. All attestation requests must pay a fee greater than zero as determined by the type-source mapping in `typeAndSourceFees`, preventing zero-fee spam attacks.

3. Merkle roots stored in the Relay contract are immutable once set for a given (protocolId, votingRoundId) pair, ensuring attestation proofs remain verifiable indefinitely.
4. The requestsOffsetSeconds must be strictly less than votingEpochDurationSeconds, ensuring the reward epoch assignment window near boundaries is valid.
5. Inflation rewards offered for a reward epoch are calculated using time-weighted distribution where $\text{totalRewardsAmount} = (\text{totalInflationReceivedWei} - \text{totalInflationRewardsOfferedWei}) * \text{rewardEpochDuration} / (\text{intervalEnd} - \text{intervalStart})$, ensuring fair proportional distribution.
6. FdcInflationConfigurations can only contain attestation types with corresponding fee configurations, validated via _checkFdcConfiguration, which calls getRequestFee and reverts if the type-source pair is not supported.
7. Verification functions return true only when both the attestationType string matches exactly (e.g., bytes32("Payment")) and the Merkle proof validates against the stored root for the specified voting round.
8. The reward epoch assignment for attestation requests follows deterministic logic. If the request occurs within requestsOffsetSeconds before the current epoch end and the next epoch's voting has started, fees are assigned to the next epoch; otherwise, they go to the current epoch.

Attack surface

The following outlines the attack surface:

1. **Fee underpayment.** This is prevented via the require(msg.value >= fee, "fee too low, call getRequestFee to get the required fee amount") check in requestAttestation, ensuring users cannot submit requests with insufficient payment.
2. **Attestation type mismatch.** This is prevented via explicit string comparison _proof.data.attestationType == bytes32("Payment") in each verification function, ensuring proofs cannot be reused across different attestation types.
3. **Balance manipulation via self-destruct.** This is prevented via the mustBalance modifier, which detects unexpected balance increases and transfers excess funds to the burn address 0x00000000000000000000000000000000dEaD.
4. **Unauthorized inflation offers.** This is prevented via the onlyFlareSystemsManager modifier on triggerRewardEpochSwitchover, ensuring only the authorized system manager can trigger reward distribution.
5. **Invalid fee configuration.** This is prevented via _checkFdcConfiguration, which validates that each FdcConfiguration has a corresponding nonzero fee entry before allowing it to be added.

6. **Double-spending of inflation.** This is prevented via the accounting invariant where `totalInflationRewardsOfferedWei` is incremented atomically when rewards are sent to the `RewardManager`, ensuring inflation cannot be distributed multiple times.

5.9. EntityManager

Description

The `EntityManager` contract serves as a central registry for voter identity and operational metadata within the Flare protocol, managing the association between voter addresses and their corresponding delegation addresses, submit addresses, submit signatures addresses, signing-policy addresses, node IDs, and public keys with full historical tracking via checkpoint-based storage. The contract implements a two-step registration pattern for address associations (propose then confirm) to ensure mutual consent, while node IDs and public keys are registered directly with cryptographic verification, and all data is queryable at any historical block number to support protocol operations like signing-policy generation and reward distribution. Shown below is the core functionality of the `EntityManager` contract:

1. **Node identity management.** It registers and unregisters node IDs with cryptographic proof of possession via `nodePossessionVerifier`, enforcing uniqueness and a configurable maximum per entity (`maxNodeIdsPerEntity`)
2. **Public-key registration.** It manages ECDSA public keys (split into two 32-byte parts) with verification via `publicKeyVerifier`, ensuring one public key per voter with automatic unregistration of old keys.
3. **Address delegation system.** It implements the propose-confirm pattern for four address types (delegation, submit, submit signatures, signing policy) where voters propose and the target address must confirm, preventing unauthorized associations.
4. **Initial setup.** Governance can set initial voter data via `setInitialVoterData` before production mode, bypassing the propose-confirm flow for bootstrapping.

Invariants

The following invariants must hold true:

1. Each node ID can only be registered to one voter at any given time, enforced by the `nodeIdRegistered` mapping check requiring `address(0)` before registration.
2. Each public-key hash can only be registered to one voter at any given time, with automatic unregistration of the previous owner's key when a voter registers a new one.
3. Each specialized address (delegation, submit, submit signatures, signing policy) can only be associated with one voter at any time, enforced by registration mappings.

4. The number of node IDs per entity never exceeds `maxNodeIdsPerEntity`, validated in `addRemoveNodeId` within the `NodesHistory` library.
5. All address associations requiring mutual consent follow the propose-confirm pattern where the voter proposes and the target address must confirm before activation.
6. Default address behavior returns the voter's own address when no specialized address is registered, implemented in all getter functions.
7. The `maxNodeIdsPerEntity` parameter can only be increased, never decreased, as enforced in `setMaxNodeIdsPerEntity`.
8. Initial voter data can only be set before production mode is activated, preventing governance from modifying voter registrations after launch.

Attack surface

The following outlines the attack surface:

1. **Node ID griefing.** This is prevented via the uniqueness check `nodeIdRegistered[_nodeId].addressAtNow() == address(0)`, which ensures a node ID cannot be registered if already claimed, and the `nodePossessionVerifier` ensures only legitimate node owners can register.
2. **Public-key collision.** This is prevented via the `keccak256(abi.encode(_part1, _part2))` hash check ensuring no two voters can register the same public key, with automatic cleanup of old keys when updating.
3. **Registration queue poisoning.** This is prevented via the confirmation checks requiring an exact match between queued address and `msg.sender`, and the uniqueness checks ensuring addresses are not already registered elsewhere.
4. **Exceeding node limits.** This is prevented via the `maxNodeIdsPerEntity` enforcement in `addRemoveNodeId`, which reverts with "Max nodes exceeded" if the limit would be breached.

5.10. Relay

Description

The Relay contract serves as a finalization layer for the FTSO (Flare Time Series Oracle) protocol, enabling decentralized verification and relay of protocol messages through weighted signature validation. It manages signing policies for reward epochs, verifies multi-signature attestations from voters, finalizes protocol messages by storing their Merkle roots, and provides random number generation.

Shown below is the core functionality of the Relay contract:

1. **Signing-policy initialization.** The `setSigningPolicy` function allows the designated `signingPolicySetter` to register new signing policies for sequential reward epochs, encoding voter addresses, weights, thresholds, and random seeds into a compact byte representation and storing its hash.
2. **Protocol message relay.** The relay function processes calldata containing a signing policy, a protocol message (with protocol ID, voting round ID, and Merkle root), and ECDSA signatures with voter indexes, verifying that accumulated signature weight exceeds the threshold before finalizing the message.
3. **Random number generation.** Provides secure random numbers derived from finalized Merkle roots of the designated random number protocol, maintaining both current and historical random values with security flags.
4. **Merkle-proof verification.** The verify function allows external callers to validate leaves against stored Merkle roots by providing a Merkle proof, with protocol-specific fees enforced.
5. **Custom signature verification.** The `verifyCustomSignature` and `governanceFeeSetup` functions enable special governance operations by verifying signatures for custom message hashes, allowing fee configuration updates through weighted signature approval.
6. **Legacy relay support.** The contract maintains backward compatibility with an `oldRelay` reference, delegating queries for historical voting rounds that predate the current contract's initialization.

Invariants

The following invariants must hold true:

1. The signing policy for reward epoch R can only be initialized after reward epoch $R - 1$ has been initialized, ensuring sequential progression of signing policies without gaps.
2. The total weight of all voters in a signing policy must be less than 2^{16} (65,536), and the threshold must be between 50% and 66% of the total weight (enforced via `MIN_THRESHOLD_BIPS` and `MAX_THRESHOLD_BIPS`).
3. A protocol message for voting round v in reward epoch $\text{exp}(v)$ can only be signed by a signing policy for reward epoch r where $\text{exp}(v) \geq r$, preventing the use of future signing policies to sign past messages.
4. When using an old signing policy (reward epoch r) to sign a message in a later reward epoch ($\text{exp}(v) > r$), the threshold is increased by `thresholdIncreaseBIPS` if r equals `lastInitializedRewardEpoch`, requiring stronger consensus for delayed signing.
5. If a newer signing policy for reward epoch $r+1$ exists with starting voting round $s+$, then the old signing policy for epoch r cannot sign messages with voting round $v \geq s+$, enforcing timely adoption of new signing policies.

6. Messages cannot be finalized if their reward epoch is more than `messageFinalizationWindowInRewardEpochs` behind the `lastInitializedRewardEpoch`, preventing indefinite influence of old signing policies.
7. Each protocol message (identified by `protocolId` and `votingRoundId`) can only be relayed once, as indicated by the nonzero Merkle root stored in `merkleRootsPrivate`.
8. Signature indexes within a relay call must be strictly increasing and within the valid voter range, preventing duplicate signature counting and ensuring deterministic weight accumulation.
9. The voting round ID of a message must be at least the `startingVotingRoundId` of the signing policy being used (except for protocol ID 1), preventing premature use of signing policies before their validity period.
10. When `signingPolicySetter` is set (mainnet deployment), signing-policy relay via `protocolId == 0` is disabled (`noSigningPolicyRelay == true`), and fees cannot be configured during construction.

Attack surface

The following outlines the attack surface:

1. **Replay attacks across protocols.** This is prevented via the inclusion of `protocolId` and `votingRoundId` in the message structure, combined with the `merkleRootsPrivate` mapping that ensures each protocol message can only be relayed once per protocol-voting round pair.
2. **Replay attacks across chains.** This is prevented via the `governanceFeeSetup` function requiring `_config.chainId == block.chainid`, ensuring governance messages are chain specific and cannot be replayed on different EVM chains.
3. **Threshold manipulation.** This is prevented via validation in both `setSigningPolicy` and `checkThresholdConsistency` that enforces threshold bounds between `MIN_THRESHOLD_BIPS` (50%) and `MAX_THRESHOLD_BIPS` (66%) relative to total weight, and the total weight must not exceed $2^{16} - 1$.
4. **Random number predictability through merkle root.** This is prevented via the design where the random number is derived from `keccak256(abi.encode(merkleRoot))` of the finalized protocol message, which can only be known after the weighted signature threshold is met and the message is finalized, making it computationally infeasible to predict or manipulate the output before finalization.
5. **Invalid voting round for protocol ID 1.** This is prevented via explicit checks that `votingRoundId` and `isSecureRandom` must both be zero when `protocolId == 1`, enforcing the special message format for custom signature verification.
6. **Calldata length manipulation.** This is prevented via multiple length checks throughout the assembly code in relay, including validation of signing-policy length, message length, signature count, and total calldata size before processing.

5.11. RewardManager

Description

The RewardManager contract is a multisource reward-distribution system that manages claim processing for protocol participants across multiple reward types (WNAT delegations, P-Chain stake mirrors, and C-Chain stakes) using Merkle-proof verification. Shown below is the core functionality of the RewardManager contract:

1. **Reward reception.** It accepts reward funds from authorized reward offers managers for future or current reward epochs, tracking both inflation and community-funded rewards separately via `receiveRewards`.
2. **Claim processing.** It validates and processes reward claims through Merkle proofs with support for five claim types (DIRECT, FEE, WNAT, MIRROR, CCHAIN), where DIRECT and FEE claims are immediate payouts while weight-based claims distribute rewards proportionally based on vote power snapshots.
3. **Weight-based distribution.** It initializes and distributes rewards proportionally across delegators, P-Chain stakers, and C-Chain stakers by calculating each share based on their vote power weight at the reward epoch's snapshot block.
4. **Autoclaiming.** It enables executors to batch-claim rewards for multiple reward owners and their PDAs in exchange for a fee, automatically wrapping rewards to WNAT.
5. **Epoch expiration.** It closes expired reward epochs by burning all unclaimed rewards and preventing further claims, with expiration triggered by `FlareSystemsManager` or a successor RewardManager contract.
6. **Burn mechanism.** It applies performance-based burn factors to FEE claims using `FlareSystemsCalculator.calculateBurnFactorPPM`, reducing rewards for underperforming voters and sending burned amounts to the burn address.
7. **Balance enforcement.** It maintains strict balance accounting through the `mustBalance` modifier, detecting and burning any unexpected funds (e.g., from self-destruct operations) to prevent balance manipulation.
8. **Executor authorization.** It validates that claim executors are authorized by reward owners via `ClaimSetupManager` and that recipients are on the allowed recipient list, enabling secure delegated claiming.
9. **Life-cycle management.** It supports contract activation/deactivation, migration from old RewardManager instances, and forwarding of epoch closure calls during transition periods.

Invariants

The following invariants must hold true:

1. The contract's actual balance must always equal `totalRewardsWei - totalClaimedWei - totalBurnedWei`, enforced by the `mustBalance` modifier on all state-changing functions to prevent balance manipulation.
2. For each reward epoch, the sum of `epochClaimedRewards`, `epochBurnedRewards`, and remaining unclaimed rewards must equal `epochTotalRewards`, ensuring no rewards are created or destroyed improperly.
3. Weight-based claims can only be processed after initialization via Merkle proof verification, and the sum of all claimed weights for a beneficiary cannot exceed their total initialized weight stored in `UnclaimedRewardState.weight`.
4. DIRECT and FEE claims can only be processed once per unique claim hash (computed as `keccak256(abi.encode(rewardClaim))`), tracked via `epochProcessedRewardClaims` to prevent double-claiming.
5. Reward epochs can only be claimed if they are past epochs (less than current epoch ID), have not expired (greater than or equal to `nextRewardEpochIdToExpire`), and have a nonzero rewards hash set by `FlareSystemsManager`.
6. The `rewardOwnerNextClaimableEpochId` for any address is monotonically increasing, ensuring that once epochs are marked as claimed, they cannot be reclaimed.
7. For weight-based claims, the reward amount calculation `amount * weight / totalWeight` must satisfy `calculatedAmount <= state.amount` and `claimedWeight <= state.weight`, preventing overdistribution.
8. The `firstClaimableRewardEpochId` can only be set once from `type(uint24).max` to the current epoch ID via `enableClaims`, establishing a permanent lower bound for claimable epochs.
9. Vote power snapshots are immutable once cached in `epochVotePowerBlock`, ensuring consistent reward calculations for all claimants within the same reward epoch.
10. Autoclaim executor fees must be less than or equal to the claimed amount for each reward owner, enforced by the `check rewardAmount >= executorFeeValue` to prevent executors from claiming when rewards are insufficient.
11. The `initialRewardEpochId` and `nextRewardEpochIdToExpire` can only be set once during initial setup via `setInitialRewardData`, establishing the starting point for epoch expiration tracking.
12. For FEE claims, the burn amount is capped at the reward amount (`Math.min(rewardAmount * burnFactor / PPM_MAX, rewardAmount)`), ensuring burns never exceed available rewards.

Attack surface

The following outlines the attack surface:

1. **Double-claiming of DIRECT/FEE rewards.** This is prevented via the `epochProcessedRewardClaims` mapping that marks each unique claim hash as processed, rejecting subsequent attempts to claim the same reward.
2. **Unauthorized claim execution.** This is prevented via the `onlyExecutorAndAllowedRecipient` modifier, which validates through `ClaimSetupManager.checkExecutorAndAllowedRecipient` that the caller is either the reward owner or an authorized executor and that the recipient is on the allowed list.
3. **Claiming future rewards.** This is prevented via `_checkIsPastRewardEpoch`, which requires `_rewardEpochId < _getCurrentRewardEpochId()`, ensuring only finalized epochs with computed rewards can be claimed.
4. **Reward epoch ID overflow.** This is prevented via `SafeCast.toUint24()` conversions throughout the codebase, ensuring epoch IDs remain within the `uint24` range and revert on overflow.
5. **Out-of-order epoch closure.** This is prevented via the check `require(nextRewardEpochIdToExpire == _rewardEpochId)` in `closeExpiredRewardEpoch`, ensuring epochs are closed sequentially to maintain consistent expiration state.

5.12. FlareSystemsManager

Description

The `FlareSystemsManager` contract orchestrates the Flare network's reward epoch life cycle by coordinating voting rounds, managing signing policies, and collecting threshold-based signatures for uptime votes and rewards. It operates as a state machine driven by `FlareDaemon` calls, progressing each reward epoch through distinct phases: random acquisition, voter registration, signing-policy initialization, uptime vote submission/signing, and rewards signing. Shown below is the core functionality of the `FlareSystemsManager` contract:

1. **Random acquisition and vote power block selection.** It acquires secure randomness from the Relay contract to select a historical vote power block for the next reward epoch, with fallback to the current epoch's vote power block if secure randomness is unavailable within the configured time/block limits.
2. **Voter registration coordination.** It enables time-bounded voter registration after vote power block selection, ensuring minimum duration requirements (both time and blocks) are met or the minimum voter threshold is reached before proceeding.
3. **Signing-policy initialization.** It creates and publishes new signing policies to the Relay contract with normalized voter weights and threshold requirements, determining which voters are eligible for the next reward epoch.
4. **Threshold-based signature collection.** It collects weighted signatures from voters for signing policies, uptime votes, and rewards, tracking accumulated weight until the threshold is reached to finalize each phase.

5. **Uptime vote management.** It manages two-phase uptime voting where voters first submit their node ID lists and then sign a consensus hash of eligible nodes after the submission period ends.
6. **Rewards data finalization.** It collects signatures for the rewards hash and weight-based claims data, enabling reward distribution once the threshold is reached.
7. **Voting-round initialization.** It initializes new voting rounds on the Submission contract with appropriate submit addresses based on current and previous reward epochs.
8. **Reward-epoch expiration.** It triggers closing of expired reward epochs on RewardManager and cleanup of old vote power blocks when configured to do so.
9. **Switchover trigger notifications.** It notifies registered trigger contracts when reward epochs switch over, enabling coordinated system-wide transitions.

Invariants

The following invariants must hold true:

1. Each reward epoch must complete random acquisition and select a vote power block before voter registration can begin, ensuring deterministic voter eligibility based on historical state.
2. Voter registration remains enabled until both minimum time and minimum block count have elapsed and either minimum voter count is reached or registration naturally closes, preventing premature signing-policy creation.
3. A signing policy must be fully signed (threshold reached) before the corresponding reward epoch can start, ensuring consensus on voter set before epoch activation.
4. Voters can only sign each message (signing policy, uptime vote hash, rewards hash) once per reward epoch, with their signature tracked via VoterData to prevent double-signing.
5. Uptime vote signing can only begin after the uptime vote submission phase ends, enforcing sequential phases.
6. Rewards can only be signed after both the uptime vote hash is finalized and the next reward epoch's signing policy is signed, ensuring proper epoch sequencing.
7. The `startVotingRoundId` for a new signing policy must be at least `currentVotingEpochId + newSigningPolicyMinNumberOfVotingRoundsDelay + 1`, preventing immediate activation and allowing time for policy propagation.
8. Reward epochs can only be closed for expiration in sequential order via `rewardEpochIdToExpireNext`, preventing gaps in the expiration sequence.
9. When the threshold is reached for any vote type, the accumulated weight is deleted to save gas, as it is no longer needed once the vote is finalized.

10. Signature recovery via ECDSA must resolve to a registered voter with nonzero weight in the VoterRegistry, ensuring only authorized participants can vote.

Attack surface

The following outlines the attack surface:

1. **Signature replay across epochs.** This is prevented via the reward epoch ID being encoded in all message hashes (signing policy, uptime vote, rewards), making signatures epoch specific and nonreplayable.
2. **Unauthorized vote submission.** This is prevented via ECDSA signature recovery in `_getVoterData`, which validates that the signer is a registered voter with nonzero weight from the VoterRegistry contract.
3. **Vote power block manipulation.** This is prevented via using secure randomness from the Relay contract when available, with the random number timestamp validated to be after acquisition start, and fallback to the previous epoch's vote power block only after maximum duration limits are exceeded.
4. **Voting on current or future epochs.** This is prevented via `_checkIfPastRewardEpoch` requiring `_rewardEpochId < _getCurrentRewardEpochId()`, ensuring votes only apply to completed epochs.

6. Assessment Results

During our assessment on the scoped FSP V1 and V2 contracts, we discovered 20 findings. No critical issues were found. Six findings were of medium impact, nine were of low impact, and the remaining findings were informational in nature.

6.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.